

A PRACTICAL, STANDARDS-AWARE GUIDE

MODERN C++

From First Program to Professional Software

Scott Brodie Forsyth

First edition – July 2026

C++23 baseline • Portable practice • Tested examples

Preface

Modern C++ is both a language and a way of making trade-offs. It can expose the machine closely, but it can also express high-level abstractions with strong compile-time guarantees. Those capabilities are useful only when the programmer can reason about types, ownership, lifetime, interfaces, failure, portability, and measurement.

This book is designed as a continuous path from a first program to professional software. It does not treat C++ as synonymous with object-oriented programming, nor does it present a single style as universally correct. Procedural, object-oriented, generic, functional, and data-oriented techniques all appear where they solve real problems.

The book is deliberately standards-aware. The selected baseline is C++23, the current published ISO revision at the time of this edition. C++26 material is discussed only as work in progress or implementation-dependent material, never as a portable requirement. The distinction matters: a compiler can implement a proposal before that proposal is part of a published standard.

How to read this book

The chapters are short and cumulative. Read the first two parts in order. From Part III onward, use the prerequisites and cross-references to move between topics. The projects are the connective tissue: they turn isolated language facilities into maintainable programs with tests, documentation, persistence, and a reproducible build.

Every complete program in the book has a corresponding file in the codebase. The build scripts are intentionally ordinary: a direct compiler command is shown first, followed by CMake when project structure becomes useful. The local verification report records exactly what was tested in the production environment.

Standards baseline and evidence

The book targets **C++23** with the compiler option `-std=c++23` (GCC/Clang) or `/std:c++23` where supported by MSVC. The current published ISO standard is ISO/IEC 14882:2024(E), informally called C++23 because the technical work was completed in 2023 [International Organization for Standardization 2024; Standard C++ Foundation 2026a]. C++26 remains a moving target in the standards process, so examples requiring draft-only facilities are excluded from the portable baseline [Standard C++ Foundation 2026b].

Support is feature-by-feature. GCC, Clang, and MSVC may implement different portions of C++23 and may expose later proposals under experimental switches. Consult the vendors' status pages before depending on a library facility, especially modules, formatting, ranges extensions, coroutines, and newer C++26 proposals [GCC Project 2026; LLVM Project 2026; Microsoft 2026].

The examples emphasize standard library facilities that are broadly implemented in current toolchains. When support is incomplete or materially different, the text labels the limitation and supplies a portable alternative. CMake is introduced as a build-system generator, not as a compiler; its role is to produce native build files for tools such as Make, Ninja, Visual Studio, and Xcode [Kitware 2026].

Terminology convention. “The standard requires” means a requirement of ISO C++. “Typically” describes common implementations. “Recommended” describes engineering guidance. “In this book” describes a pedagogical or stylistic choice. These are not interchangeable claims.

Notation and safety conventions

Code blocks labelled **wrong** are deliberately incorrect and are never used as build inputs. Undefined behavior is marked explicitly. A successful compilation is not proof that a program is correct: the compiler checks a language-defined subset of properties, while tests, analysis, review, and measurement address the rest.

Mathematics appears only when it clarifies a model. Whenever notation is introduced, the symbols are defined in ordinary language before they are used. Complexity claims are separated from measurements, and claims about performance are tied to a documented environment.

Roadmap

- Part I** Computers, tools, and the translation process.
- Part II** Core language foundations and program control.
- Part III** Data representation, memory, ownership, and lifetime.
- Part IV** User-defined types and competing design styles.
- Part V** Containers, algorithms, ranges, files, and library facilities.
- Part VI** Templates, concepts, and compile-time programming.
- Part VII** Errors, testing, debugging, and reliability.
- Part VIII** Data structures, algorithms, complexity, and benchmarking.
- Part IX** Build systems, APIs, portability, and professional practice.
- Part X** Concurrency and the C++ memory model.
- Part XI** Performance, profiling, locality, and systems trade-offs.
- Part XII** Advanced interfaces, metaprogramming, coroutines, and architecture.

Contents

Preface	ii
Standards baseline and evidence	iii
Notation and safety conventions	iv
Roadmap	v
I Computers and Programming	1
Part 1: Computers and Programming	2
1 What a Computer Program Is	3
1.1 The problem and the mental model	3
1.2 A small working program	3
1.3 Rules, mistakes, and diagnostics	3
1.4 Design, performance, and testing	4
1.5 Exercises	4
1.6 Summary and further study	4
2 Hardware, Software, and Operating Systems	5
2.1 The problem and the mental model	5
2.2 A small working program	5
2.3 Rules, mistakes, and diagnostics	5
2.4 Design, performance, and testing	6
2.5 Exercises	6
2.6 Summary and further study	6
3 Source Code, Machine Code, and Executable Files	7
3.1 The problem and the mental model	7
3.2 A small working program	7
3.3 Rules, mistakes, and diagnostics	7
3.4 Design, performance, and testing	8
3.5 Exercises	8
3.6 Summary and further study	8
4 Compilers, Linkers, Libraries, and Build Processes	9
4.1 The problem and the mental model	9
4.2 A small working program	9
4.3 Rules, mistakes, and diagnostics	9
4.4 Design, performance, and testing	10
4.5 Exercises	10
4.6 Summary and further study	10

5	Installing a C++ Development Environment	11
5.1	The problem and the mental model	11
5.2	A small working program	11
5.3	Rules, mistakes, and diagnostics	11
5.4	Design, performance, and testing	12
5.5	Exercises	12
5.6	Summary and further study	12
6	Terminals, Editors, IDEs, and File Systems	13
6.1	The problem and the mental model	13
6.2	A small working program	13
6.3	Rules, mistakes, and diagnostics	13
6.4	Design, performance, and testing	14
6.5	Exercises	14
6.6	Summary and further study	14
7	Writing, Compiling, and Running the First Program	15
7.1	The problem and the mental model	15
7.2	A small working program	15
7.3	Rules, mistakes, and diagnostics	15
7.4	Design, performance, and testing	16
7.5	Exercises	16
7.6	Summary and further study	16
8	Reading Compiler Diagnostics	17
8.1	The problem and the mental model	17
8.2	A small working program	17
8.3	Rules, mistakes, and diagnostics	17
8.4	Design, performance, and testing	18
8.5	Exercises	18
8.6	Summary and further study	18
II	The Foundations of C++	19
Part 2:	The Foundations of C++	20
9	Program Structure	21
9.1	The problem and the mental model	21
9.2	A small working program	21
9.3	Rules, mistakes, and diagnostics	21
9.4	Design, performance, and testing	22
9.5	Exercises	22
9.6	Summary and further study	22
10	Statements, Expressions, and Blocks	23
10.1	The problem and the mental model	23
10.2	A small working program	23
10.3	Rules, mistakes, and diagnostics	23
10.4	Design, performance, and testing	24
10.5	Exercises	24
10.6	Summary and further study	24

11	Variables and Initialization	25
11.1	The problem and the mental model	25
11.2	A small working program	25
11.3	Rules, mistakes, and diagnostics	25
11.4	Design, performance, and testing	26
11.5	Exercises	26
11.6	Summary and further study	26
12	Fundamental Types	27
12.1	The problem and the mental model	27
12.2	A small working program	27
12.3	Rules, mistakes, and diagnostics	27
12.4	Design, performance, and testing	28
12.5	Exercises	28
12.6	Summary and further study	28
13	Literals	29
13.1	The problem and the mental model	29
13.2	A small working program	29
13.3	Rules, mistakes, and diagnostics	29
13.4	Design, performance, and testing	30
13.5	Exercises	30
13.6	Summary and further study	30
14	Arithmetic and Operators	31
14.1	The problem and the mental model	31
14.2	A small working program	31
14.3	Rules, mistakes, and diagnostics	31
14.4	Design, performance, and testing	32
14.5	Exercises	32
14.6	Summary and further study	32
15	Comparisons and Boolean Logic	33
15.1	The problem and the mental model	33
15.2	A small working program	33
15.3	Rules, mistakes, and diagnostics	33
15.4	Design, performance, and testing	34
15.5	Exercises	34
15.6	Summary and further study	34
16	Input and Output	35
16.1	The problem and the mental model	35
16.2	A small working program	35
16.3	Rules, mistakes, and diagnostics	35
16.4	Design, performance, and testing	36
16.5	Exercises	36
16.6	Summary and further study	36
17	Selection with if and switch	37
17.1	The problem and the mental model	37
17.2	A small working program	37
17.3	Rules, mistakes, and diagnostics	37
17.4	Design, performance, and testing	38

17.5	Exercises	38
17.6	Summary and further study	38
18	Repetition with for, while, and do-while	39
18.1	The problem and the mental model	39
18.2	A small working program	39
18.3	Rules, mistakes, and diagnostics	39
18.4	Design, performance, and testing	40
18.5	Exercises	40
18.6	Summary and further study	40
19	Scope, Lifetime, and Name Visibility	41
19.1	The problem and the mental model	41
19.2	A small working program	41
19.3	Rules, mistakes, and diagnostics	41
19.4	Design, performance, and testing	42
19.5	Exercises	42
19.6	Summary and further study	42
20	Functions	43
20.1	The problem and the mental model	43
20.2	A small working program	43
20.3	Rules, mistakes, and diagnostics	43
20.4	Design, performance, and testing	44
20.5	Exercises	44
20.6	Summary and further study	44
21	Function Parameters and Return Values	45
21.1	The problem and the mental model	45
21.2	A small working program	45
21.3	Rules, mistakes, and diagnostics	45
21.4	Design, performance, and testing	46
21.5	Exercises	46
21.6	Summary and further study	46
22	Overloading and Default Arguments	47
22.1	The problem and the mental model	47
22.2	A small working program	47
22.3	Rules, mistakes, and diagnostics	47
22.4	Design, performance, and testing	48
22.5	Exercises	48
22.6	Summary and further study	48
23	Type Deduction with auto	49
23.1	The problem and the mental model	49
23.2	A small working program	49
23.3	Rules, mistakes, and diagnostics	49
23.4	Design, performance, and testing	50
23.5	Exercises	50
23.6	Summary and further study	50
24	Constants and constexpr	51
24.1	The problem and the mental model	51

24.2	A small working program	51
24.3	Rules, mistakes, and diagnostics	51
24.4	Design, performance, and testing	52
24.5	Exercises	52
24.6	Summary and further study	52
25	Enumerations	53
25.1	The problem and the mental model	53
25.2	A small working program	53
25.3	Rules, mistakes, and diagnostics	53
25.4	Design, performance, and testing	54
25.5	Exercises	54
25.6	Summary and further study	54
26	Namespaces	55
26.1	The problem and the mental model	55
26.2	A small working program	55
26.3	Rules, mistakes, and diagnostics	55
26.4	Design, performance, and testing	56
26.5	Exercises	56
26.6	Summary and further study	56
III	Data and Memory	57
Part 3:	Data and Memory	58
27	Binary Representation	59
27.1	The problem and the mental model	59
27.2	A small working program	59
27.3	Rules, mistakes, and diagnostics	59
27.4	Design, performance, and testing	60
27.5	Exercises	60
27.6	Summary and further study	60
28	Integers, Floating-Point Values, and Precision	61
28.1	The problem and the mental model	61
28.2	A small working program	61
28.3	Rules, mistakes, and diagnostics	61
28.4	Design, performance, and testing	62
28.5	Exercises	62
28.6	Summary and further study	62
29	Characters, Text, Encoding, and Unicode	63
29.1	The problem and the mental model	63
29.2	A small working program	63
29.3	Rules, mistakes, and diagnostics	63
29.4	Design, performance, and testing	64
29.5	Exercises	64
29.6	Summary and further study	64
30	Arrays	65
30.1	The problem and the mental model	65

30.2	A small working program	65
30.3	Rules, mistakes, and diagnostics	65
30.4	Design, performance, and testing	66
30.5	Exercises	66
30.6	Summary and further study	66
31	std::array	67
31.1	The problem and the mental model	67
31.2	A small working program	67
31.3	Rules, mistakes, and diagnostics	67
31.4	Design, performance, and testing	68
31.5	Exercises	68
31.6	Summary and further study	68
32	std::vector	69
32.1	The problem and the mental model	69
32.2	A small working program	69
32.3	Rules, mistakes, and diagnostics	69
32.4	Design, performance, and testing	70
32.5	Exercises	70
32.6	Summary and further study	70
33	std::string and std::string_view	71
33.1	The problem and the mental model	71
33.2	A small working program	71
33.3	Rules, mistakes, and diagnostics	71
33.4	Design, performance, and testing	72
33.5	Exercises	72
33.6	Summary and further study	72
34	References	73
34.1	The problem and the mental model	73
34.2	A small working program	73
34.3	Rules, mistakes, and diagnostics	73
34.4	Design, performance, and testing	74
34.5	Exercises	74
34.6	Summary and further study	74
35	Pointers	75
35.1	The problem and the mental model	75
35.2	A small working program	75
35.3	Rules, mistakes, and diagnostics	75
35.4	Design, performance, and testing	76
35.5	Exercises	76
35.6	Summary and further study	76
36	Null Pointers	77
36.1	The problem and the mental model	77
36.2	A small working program	77
36.3	Rules, mistakes, and diagnostics	77
36.4	Design, performance, and testing	78
36.5	Exercises	78
36.6	Summary and further study	78

37	Dynamic Storage	79
37.1	The problem and the mental model	79
37.2	A small working program	79
37.3	Rules, mistakes, and diagnostics	79
37.4	Design, performance, and testing	80
37.5	Exercises	80
37.6	Summary and further study	80
38	Stack and Free-Store Concepts	81
38.1	The problem and the mental model	81
38.2	A small working program	81
38.3	Rules, mistakes, and diagnostics	81
38.4	Design, performance, and testing	82
38.5	Exercises	82
38.6	Summary and further study	82
39	Object Lifetime	83
39.1	The problem and the mental model	83
39.2	A small working program	83
39.3	Rules, mistakes, and diagnostics	83
39.4	Design, performance, and testing	84
39.5	Exercises	84
39.6	Summary and further study	84
40	Resource Ownership	85
40.1	The problem and the mental model	85
40.2	A small working program	85
40.3	Rules, mistakes, and diagnostics	85
40.4	Design, performance, and testing	86
40.5	Exercises	86
40.6	Summary and further study	86
41	Smart Pointers	87
41.1	The problem and the mental model	87
41.2	A small working program	87
41.3	Rules, mistakes, and diagnostics	87
41.4	Design, performance, and testing	88
41.5	Exercises	88
41.6	Summary and further study	88
42	RAII	89
42.1	The problem and the mental model	89
42.2	A small working program	89
42.3	Rules, mistakes, and diagnostics	90
42.4	Design, performance, and testing	90
42.5	Exercises	90
42.6	Summary and further study	90
43	Copy and Move Semantics	91
43.1	The problem and the mental model	91
43.2	A small working program	91
43.3	Rules, mistakes, and diagnostics	91
43.4	Design, performance, and testing	92

43.5	Exercises	92
43.6	Summary and further study	92
44	Value Categories	93
44.1	The problem and the mental model	93
44.2	A small working program	93
44.3	Rules, mistakes, and diagnostics	93
44.4	Design, performance, and testing	94
44.5	Exercises	94
44.6	Summary and further study	94
45	Undefined, Unspecified, and Implementation-Defined Behavior	95
45.1	The problem and the mental model	95
45.2	A small working program	95
45.3	Rules, mistakes, and diagnostics	95
45.4	Design, performance, and testing	96
45.5	Exercises	96
45.6	Summary and further study	96
IV	User-Defined Types	97
Part 4:	User-Defined Types	98
46	Structures	99
46.1	The problem and the mental model	99
46.2	A small working program	99
46.3	Rules, mistakes, and diagnostics	99
46.4	Design, performance, and testing	100
46.5	Exercises	100
46.6	Summary and further study	100
47	Classes and Objects	101
47.1	The problem and the mental model	101
47.2	A small working program	101
47.3	Rules, mistakes, and diagnostics	102
47.4	Design, performance, and testing	102
47.5	Exercises	102
47.6	Summary and further study	102
48	Access Control	103
48.1	The problem and the mental model	103
48.2	A small working program	103
48.3	Rules, mistakes, and diagnostics	103
48.4	Design, performance, and testing	104
48.5	Exercises	104
48.6	Summary and further study	104
49	Constructors	105
49.1	The problem and the mental model	105
49.2	A small working program	105
49.3	Rules, mistakes, and diagnostics	105
49.4	Design, performance, and testing	106

49.5	Exercises	106
49.6	Summary and further study	106
50	Destructors	107
50.1	The problem and the mental model	107
50.2	A small working program	107
50.3	Rules, mistakes, and diagnostics	107
50.4	Design, performance, and testing	108
50.5	Exercises	108
50.6	Summary and further study	108
51	Invariants	109
51.1	The problem and the mental model	109
51.2	A small working program	109
51.3	Rules, mistakes, and diagnostics	109
51.4	Design, performance, and testing	110
51.5	Exercises	110
51.6	Summary and further study	110
52	Member Functions	111
52.1	The problem and the mental model	111
52.2	A small working program	111
52.3	Rules, mistakes, and diagnostics	111
52.4	Design, performance, and testing	112
52.5	Exercises	112
52.6	Summary and further study	112
53	const Correctness	113
53.1	The problem and the mental model	113
53.2	A small working program	113
53.3	Rules, mistakes, and diagnostics	113
53.4	Design, performance, and testing	114
53.5	Exercises	114
53.6	Summary and further study	114
54	Static Members	115
54.1	The problem and the mental model	115
54.2	A small working program	115
54.3	Rules, mistakes, and diagnostics	115
54.4	Design, performance, and testing	116
54.5	Exercises	116
54.6	Summary and further study	116
55	Operator Overloading	117
55.1	The problem and the mental model	117
55.2	A small working program	117
55.3	Rules, mistakes, and diagnostics	117
55.4	Design, performance, and testing	118
55.5	Exercises	118
55.6	Summary and further study	118
56	Composition	119
56.1	The problem and the mental model	119

56.2	A small working program	119
56.3	Rules, mistakes, and diagnostics	119
56.4	Design, performance, and testing	120
56.5	Exercises	120
56.6	Summary and further study	120
57	Inheritance	121
57.1	The problem and the mental model	121
57.2	A small working program	121
57.3	Rules, mistakes, and diagnostics	121
57.4	Design, performance, and testing	122
57.5	Exercises	122
57.6	Summary and further study	122
58	Runtime Polymorphism	123
58.1	The problem and the mental model	123
58.2	A small working program	123
58.3	Rules, mistakes, and diagnostics	124
58.4	Design, performance, and testing	124
58.5	Exercises	124
58.6	Summary and further study	124
59	Abstract Interfaces	125
59.1	The problem and the mental model	125
59.2	A small working program	125
59.3	Rules, mistakes, and diagnostics	125
59.4	Design, performance, and testing	126
59.5	Exercises	126
59.6	Summary and further study	126
60	Virtual Functions	127
60.1	The problem and the mental model	127
60.2	A small working program	127
60.3	Rules, mistakes, and diagnostics	127
60.4	Design, performance, and testing	128
60.5	Exercises	128
60.6	Summary and further study	128
61	Object Slicing	129
61.1	The problem and the mental model	129
61.2	A small working program	129
61.3	Rules, mistakes, and diagnostics	129
61.4	Design, performance, and testing	130
61.5	Exercises	130
61.6	Summary and further study	130
62	Multiple Inheritance	131
62.1	The problem and the mental model	131
62.2	A small working program	131
62.3	Rules, mistakes, and diagnostics	131
62.4	Design, performance, and testing	132
62.5	Exercises	132
62.6	Summary and further study	132

63	Alternatives to Inheritance	133
63.1	The problem and the mental model	133
63.2	A small working program	133
63.3	Rules, mistakes, and diagnostics	133
63.4	Design, performance, and testing	134
63.5	Exercises	134
63.6	Summary and further study	134
V	The Standard Library	135
Part 5:	The Standard Library	136
64	The Purpose and Structure of the Standard Library	137
64.1	The problem and the mental model	137
64.2	A small working program	137
64.3	Rules, mistakes, and diagnostics	137
64.4	Design, performance, and testing	138
64.5	Exercises	138
64.6	Summary and further study	138
65	Containers	139
65.1	The problem and the mental model	139
65.2	A small working program	139
65.3	Rules, mistakes, and diagnostics	139
65.4	Design, performance, and testing	140
65.5	Exercises	140
65.6	Summary and further study	140
66	Iterators	141
66.1	The problem and the mental model	141
66.2	A small working program	141
66.3	Rules, mistakes, and diagnostics	141
66.4	Design, performance, and testing	142
66.5	Exercises	142
66.6	Summary and further study	142
67	Algorithms	143
67.1	The problem and the mental model	143
67.2	A small working program	143
67.3	Rules, mistakes, and diagnostics	143
67.4	Design, performance, and testing	144
67.5	Exercises	144
67.6	Summary and further study	144
68	Ranges and Views	145
68.1	The problem and the mental model	145
68.2	A small working program	145
68.3	Rules, mistakes, and diagnostics	145
68.4	Design, performance, and testing	146
68.5	Exercises	146
68.6	Summary and further study	146

69	Function Objects	147
69.1	The problem and the mental model	147
69.2	A small working program	147
69.3	Rules, mistakes, and diagnostics	147
69.4	Design, performance, and testing	148
69.5	Exercises	148
69.6	Summary and further study	148
70	Lambda Expressions	149
70.1	The problem and the mental model	149
70.2	A small working program	149
70.3	Rules, mistakes, and diagnostics	149
70.4	Design, performance, and testing	150
70.5	Exercises	150
70.6	Summary and further study	150
71	Associative Containers	151
71.1	The problem and the mental model	151
71.2	A small working program	151
71.3	Rules, mistakes, and diagnostics	151
71.4	Design, performance, and testing	152
71.5	Exercises	152
71.6	Summary and further study	152
72	Unordered Containers	153
72.1	The problem and the mental model	153
72.2	A small working program	153
72.3	Rules, mistakes, and diagnostics	153
72.4	Design, performance, and testing	154
72.5	Exercises	154
72.6	Summary and further study	154
73	Stacks, Queues, and Priority Queues	155
73.1	The problem and the mental model	155
73.2	A small working program	155
73.3	Rules, mistakes, and diagnostics	155
73.4	Design, performance, and testing	156
73.5	Exercises	156
73.6	Summary and further study	156
74	Optional Values and Variants	157
74.1	The problem and the mental model	157
74.2	A small working program	157
74.3	Rules, mistakes, and diagnostics	157
74.4	Design, performance, and testing	158
74.5	Exercises	158
74.6	Summary and further study	158
75	Tuples	159
75.1	The problem and the mental model	159
75.2	A small working program	159
75.3	Rules, mistakes, and diagnostics	159
75.4	Design, performance, and testing	160

75.5	Exercises	160
75.6	Summary and further study	160
76	Time and Clocks	161
76.1	The problem and the mental model	161
76.2	A small working program	161
76.3	Rules, mistakes, and diagnostics	161
76.4	Design, performance, and testing	162
76.5	Exercises	162
76.6	Summary and further study	162
77	Random-Number Facilities	163
77.1	The problem and the mental model	163
77.2	A small working program	163
77.3	Rules, mistakes, and diagnostics	163
77.4	Design, performance, and testing	164
77.5	Exercises	164
77.6	Summary and further study	164
78	File Input and Output	165
78.1	The problem and the mental model	165
78.2	A small working program	165
78.3	Rules, mistakes, and diagnostics	165
78.4	Design, performance, and testing	166
78.5	Exercises	166
78.6	Summary and further study	166
79	Formatting	167
79.1	The problem and the mental model	167
79.2	A small working program	167
79.3	Rules, mistakes, and diagnostics	167
79.4	Design, performance, and testing	168
79.5	Exercises	168
79.6	Summary and further study	168
80	Filesystem Operations	169
80.1	The problem and the mental model	169
80.2	A small working program	169
80.3	Rules, mistakes, and diagnostics	169
80.4	Design, performance, and testing	170
80.5	Exercises	170
80.6	Summary and further study	170
VI	Generic Programming	171
Part 6:	Generic Programming	172
81	Function Templates	173
81.1	The problem and the mental model	173
81.2	A small working program	173
81.3	Rules, mistakes, and diagnostics	173
81.4	Design, performance, and testing	174

81.5	Exercises	174
81.6	Summary and further study	174
82	Class Templates	175
82.1	The problem and the mental model	175
82.2	A small working program	175
82.3	Rules, mistakes, and diagnostics	175
82.4	Design, performance, and testing	176
82.5	Exercises	176
82.6	Summary and further study	176
83	Template Argument Deduction	177
83.1	The problem and the mental model	177
83.2	A small working program	177
83.3	Rules, mistakes, and diagnostics	177
83.4	Design, performance, and testing	178
83.5	Exercises	178
83.6	Summary and further study	178
84	Concepts and Constraints	179
84.1	The problem and the mental model	179
84.2	A small working program	179
84.3	Rules, mistakes, and diagnostics	179
84.4	Design, performance, and testing	180
84.5	Exercises	180
84.6	Summary and further study	180
85	Variadic Templates	181
85.1	The problem and the mental model	181
85.2	A small working program	181
85.3	Rules, mistakes, and diagnostics	181
85.4	Design, performance, and testing	182
85.5	Exercises	182
85.6	Summary and further study	182
86	Compile-Time Programming	183
86.1	The problem and the mental model	183
86.2	A small working program	183
86.3	Rules, mistakes, and diagnostics	183
86.4	Design, performance, and testing	184
86.5	Exercises	184
86.6	Summary and further study	184
87	constexpr Evaluation	185
87.1	The problem and the mental model	185
87.2	A small working program	185
87.3	Rules, mistakes, and diagnostics	185
87.4	Design, performance, and testing	186
87.5	Exercises	186
87.6	Summary and further study	186
88	Type Traits	187
88.1	The problem and the mental model	187

88.2	A small working program	187
88.3	Rules, mistakes, and diagnostics	187
88.4	Design, performance, and testing	188
88.5	Exercises	188
88.6	Summary and further study	188
89	Generic Algorithms	189
89.1	The problem and the mental model	189
89.2	A small working program	189
89.3	Rules, mistakes, and diagnostics	189
89.4	Design, performance, and testing	190
89.5	Exercises	190
89.6	Summary and further study	190
90	Template Errors and Diagnostics	191
90.1	The problem and the mental model	191
90.2	A small working program	191
90.3	Rules, mistakes, and diagnostics	191
90.4	Design, performance, and testing	192
90.5	Exercises	192
90.6	Summary and further study	192
VII	Errors, Testing, and Reliability	193
Part 7:	Errors, Testing, and Reliability	194
91	Syntax, Linker, Logic, and Runtime Errors	195
91.1	The problem and the mental model	195
91.2	A small working program	195
91.3	Rules, mistakes, and diagnostics	195
91.4	Design, performance, and testing	196
91.5	Exercises	196
91.6	Summary and further study	196
92	Assertions	197
92.1	The problem and the mental model	197
92.2	A small working program	197
92.3	Rules, mistakes, and diagnostics	197
92.4	Design, performance, and testing	198
92.5	Exercises	198
92.6	Summary and further study	198
93	Exceptions	199
93.1	The problem and the mental model	199
93.2	A small working program	199
93.3	Rules, mistakes, and diagnostics	199
93.4	Design, performance, and testing	200
93.5	Exercises	200
93.6	Summary and further study	200
94	Error Codes	201
94.1	The problem and the mental model	201

94.2	A small working program	201
94.3	Rules, mistakes, and diagnostics	201
94.4	Design, performance, and testing	202
94.5	Exercises	202
94.6	Summary and further study	202
95	Expected-Style Error Handling	203
95.1	The problem and the mental model	203
95.2	A small working program	203
95.3	Rules, mistakes, and diagnostics	204
95.4	Design, performance, and testing	204
95.5	Exercises	204
95.6	Summary and further study	204
96	Exception Safety	205
96.1	The problem and the mental model	205
96.2	A small working program	205
96.3	Rules, mistakes, and diagnostics	205
96.4	Design, performance, and testing	206
96.5	Exercises	206
96.6	Summary and further study	206
97	Unit Testing	207
97.1	The problem and the mental model	207
97.2	A small working program	207
97.3	Rules, mistakes, and diagnostics	207
97.4	Design, performance, and testing	208
97.5	Exercises	208
97.6	Summary and further study	208
98	Integration Testing	209
98.1	The problem and the mental model	209
98.2	A small working program	209
98.3	Rules, mistakes, and diagnostics	209
98.4	Design, performance, and testing	210
98.5	Exercises	210
98.6	Summary and further study	210
99	Property-Based Testing	211
99.1	The problem and the mental model	211
99.2	A small working program	211
99.3	Rules, mistakes, and diagnostics	211
99.4	Design, performance, and testing	212
99.5	Exercises	212
99.6	Summary and further study	212
100	Test Doubles	213
100.1	The problem and the mental model	213
100.2	A small working program	213
100.3	Rules, mistakes, and diagnostics	213
100.4	Design, performance, and testing	214
100.5	Exercises	214
100.6	Summary and further study	214

101	Debuggers	215
101.1	The problem and the mental model	215
101.2	A small working program	215
101.3	Rules, mistakes, and diagnostics	215
101.4	Design, performance, and testing	216
101.5	Exercises	216
101.6	Summary and further study	216
102	Sanitizers	217
102.1	The problem and the mental model	217
102.2	A small working program	217
102.3	Rules, mistakes, and diagnostics	217
102.4	Design, performance, and testing	218
102.5	Exercises	218
102.6	Summary and further study	218
103	Static Analysis	219
103.1	The problem and the mental model	219
103.2	A small working program	219
103.3	Rules, mistakes, and diagnostics	219
103.4	Design, performance, and testing	220
103.5	Exercises	220
103.6	Summary and further study	220
104	Logging	221
104.1	The problem and the mental model	221
104.2	A small working program	221
104.3	Rules, mistakes, and diagnostics	221
104.4	Design, performance, and testing	222
104.5	Exercises	222
104.6	Summary and further study	222
105	Reproducing and Minimizing Bugs	223
105.1	The problem and the mental model	223
105.2	A small working program	223
105.3	Rules, mistakes, and diagnostics	223
105.4	Design, performance, and testing	224
105.5	Exercises	224
105.6	Summary and further study	224
106	Defensive Programming	225
106.1	The problem and the mental model	225
106.2	A small working program	225
106.3	Rules, mistakes, and diagnostics	225
106.4	Design, performance, and testing	226
106.5	Exercises	226
106.6	Summary and further study	226
VIII	Data Structures and Algorithms	227
Part 8:	Data Structures and Algorithms	228

107	Algorithmic Thinking	229
107.1	The problem and the mental model	229
107.2	A small working program	229
107.3	Rules, mistakes, and diagnostics	229
107.4	Design, performance, and testing	230
107.5	Exercises	230
107.6	Summary and further study	230
108	Measuring Time and Space	231
108.1	The problem and the mental model	231
108.2	A small working program	231
108.3	Rules, mistakes, and diagnostics	231
108.4	Design, performance, and testing	232
108.5	Exercises	232
108.6	Summary and further study	232
109	Asymptotic Complexity	233
109.1	The problem and the mental model	233
109.2	A small working program	233
109.3	Rules, mistakes, and diagnostics	233
109.4	Design, performance, and testing	234
109.5	Exercises	234
109.6	Summary and further study	234
110	Recursion	235
110.1	The problem and the mental model	235
110.2	A small working program	235
110.3	Rules, mistakes, and diagnostics	235
110.4	Design, performance, and testing	236
110.5	Exercises	236
110.6	Summary and further study	236
111	Searching	237
111.1	The problem and the mental model	237
111.2	A small working program	237
111.3	Rules, mistakes, and diagnostics	237
111.4	Design, performance, and testing	238
111.5	Exercises	238
111.6	Summary and further study	238
112	Sorting	239
112.1	The problem and the mental model	239
112.2	A small working program	239
112.3	Rules, mistakes, and diagnostics	239
112.4	Design, performance, and testing	240
112.5	Exercises	240
112.6	Summary and further study	240
113	Linked Structures	241
113.1	The problem and the mental model	241
113.2	A small working program	241
113.3	Rules, mistakes, and diagnostics	241
113.4	Design, performance, and testing	242

113.5	Exercises	242
113.6	Summary and further study	242
114	Stacks and Queues	243
114.1	The problem and the mental model	243
114.2	A small working program	243
114.3	Rules, mistakes, and diagnostics	243
114.4	Design, performance, and testing	244
114.5	Exercises	244
114.6	Summary and further study	244
115	Trees	245
115.1	The problem and the mental model	245
115.2	A small working program	245
115.3	Rules, mistakes, and diagnostics	245
115.4	Design, performance, and testing	246
115.5	Exercises	246
115.6	Summary and further study	246
116	Heaps	247
116.1	The problem and the mental model	247
116.2	A small working program	247
116.3	Rules, mistakes, and diagnostics	247
116.4	Design, performance, and testing	248
116.5	Exercises	248
116.6	Summary and further study	248
117	Hash Tables	249
117.1	The problem and the mental model	249
117.2	A small working program	249
117.3	Rules, mistakes, and diagnostics	249
117.4	Design, performance, and testing	250
117.5	Exercises	250
117.6	Summary and further study	250
118	Graphs	251
118.1	The problem and the mental model	251
118.2	A small working program	251
118.3	Rules, mistakes, and diagnostics	252
118.4	Design, performance, and testing	252
118.5	Exercises	252
118.6	Summary and further study	252
119	Dynamic Programming	253
119.1	The problem and the mental model	253
119.2	A small working program	253
119.3	Rules, mistakes, and diagnostics	253
119.4	Design, performance, and testing	254
119.5	Exercises	254
119.6	Summary and further study	254
120	Greedy Algorithms	255
120.1	The problem and the mental model	255

120.2	A small working program	255
120.3	Rules, mistakes, and diagnostics	255
120.4	Design, performance, and testing	256
120.5	Exercises	256
120.6	Summary and further study	256
121	Algorithm Selection	257
121.1	The problem and the mental model	257
121.2	A small working program	257
121.3	Rules, mistakes, and diagnostics	257
121.4	Design, performance, and testing	258
121.5	Exercises	258
121.6	Summary and further study	258
122	Benchmarking	259
122.1	The problem and the mental model	259
122.2	A small working program	259
122.3	Rules, mistakes, and diagnostics	259
122.4	Design, performance, and testing	260
122.5	Exercises	260
122.6	Summary and further study	260
IX	Professional C++ Development	261
Part 9:	Professional C++ Development	262
123	Organizing Source and Header Files	263
123.1	The problem and the mental model	263
123.2	A small working program	263
123.3	Rules, mistakes, and diagnostics	263
123.4	Design, performance, and testing	264
123.5	Exercises	264
123.6	Summary and further study	264
124	One Definition Rule	265
124.1	The problem and the mental model	265
124.2	A small working program	265
124.3	Rules, mistakes, and diagnostics	265
124.4	Design, performance, and testing	266
124.5	Exercises	266
124.6	Summary and further study	266
125	Separate Compilation	267
125.1	The problem and the mental model	267
125.2	A small working program	267
125.3	Rules, mistakes, and diagnostics	267
125.4	Design, performance, and testing	268
125.5	Exercises	268
125.6	Summary and further study	268
126	Libraries	269
126.1	The problem and the mental model	269

126.2	A small working program	269
126.3	Rules, mistakes, and diagnostics	269
126.4	Design, performance, and testing	270
126.5	Exercises	270
126.6	Summary and further study	270
127	Modules and Their Support Status	271
127.1	The problem and the mental model	271
127.2	A small working program	271
127.3	Rules, mistakes, and diagnostics	271
127.4	Design, performance, and testing	272
127.5	Exercises	272
127.6	Summary and further study	272
128	CMake	273
128.1	The problem and the mental model	273
128.2	A small working program	273
128.3	Rules, mistakes, and diagnostics	273
128.4	Design, performance, and testing	274
128.5	Exercises	274
128.6	Summary and further study	274
129	Dependency Management	275
129.1	The problem and the mental model	275
129.2	A small working program	275
129.3	Rules, mistakes, and diagnostics	275
129.4	Design, performance, and testing	276
129.5	Exercises	276
129.6	Summary and further study	276
130	Package Managers	277
130.1	The problem and the mental model	277
130.2	A small working program	277
130.3	Rules, mistakes, and diagnostics	277
130.4	Design, performance, and testing	278
130.5	Exercises	278
130.6	Summary and further study	278
131	Version Control with Git	279
131.1	The problem and the mental model	279
131.2	A small working program	279
131.3	Rules, mistakes, and diagnostics	279
131.4	Design, performance, and testing	280
131.5	Exercises	280
131.6	Summary and further study	280
132	Documentation	281
132.1	The problem and the mental model	281
132.2	A small working program	281
132.3	Rules, mistakes, and diagnostics	281
132.4	Design, performance, and testing	282
132.5	Exercises	282
132.6	Summary and further study	282

133	API Design	283
133.1	The problem and the mental model	283
133.2	A small working program	283
133.3	Rules, mistakes, and diagnostics	283
133.4	Design, performance, and testing	284
133.5	Exercises	284
133.6	Summary and further study	284
134	Coding Conventions	285
134.1	The problem and the mental model	285
134.2	A small working program	285
134.3	Rules, mistakes, and diagnostics	285
134.4	Design, performance, and testing	286
134.5	Exercises	286
134.6	Summary and further study	286
135	Refactoring	287
135.1	The problem and the mental model	287
135.2	A small working program	287
135.3	Rules, mistakes, and diagnostics	287
135.4	Design, performance, and testing	288
135.5	Exercises	288
135.6	Summary and further study	288
136	Code Review	289
136.1	The problem and the mental model	289
136.2	A small working program	289
136.3	Rules, mistakes, and diagnostics	289
136.4	Design, performance, and testing	290
136.5	Exercises	290
136.6	Summary and further study	290
137	Continuous Integration	291
137.1	The problem and the mental model	291
137.2	A small working program	291
137.3	Rules, mistakes, and diagnostics	291
137.4	Design, performance, and testing	292
137.5	Exercises	292
137.6	Summary and further study	292
138	Portability	293
138.1	The problem and the mental model	293
138.2	A small working program	293
138.3	Rules, mistakes, and diagnostics	293
138.4	Design, performance, and testing	294
138.5	Exercises	294
138.6	Summary and further study	294
139	Cross-Platform Development	295
139.1	The problem and the mental model	295
139.2	A small working program	295
139.3	Rules, mistakes, and diagnostics	295
139.4	Design, performance, and testing	296

139.5	Exercises	296
139.6	Summary and further study	296
140	Application Binary Interfaces	297
140.1	The problem and the mental model	297
140.2	A small working program	297
140.3	Rules, mistakes, and diagnostics	297
140.4	Design, performance, and testing	298
140.5	Exercises	298
140.6	Summary and further study	298
141	Interoperability with C	299
141.1	The problem and the mental model	299
141.2	A small working program	299
141.3	Rules, mistakes, and diagnostics	299
141.4	Design, performance, and testing	300
141.5	Exercises	300
141.6	Summary and further study	300
142	Interoperability with Other Languages	301
142.1	The problem and the mental model	301
142.2	A small working program	301
142.3	Rules, mistakes, and diagnostics	301
142.4	Design, performance, and testing	302
142.5	Exercises	302
142.6	Summary and further study	302
X	Concurrency	303
Part 10:	Concurrency	304
143	Processes and Threads	305
143.1	The problem and the mental model	305
143.2	A small working program	305
143.3	Rules, mistakes, and diagnostics	305
143.4	Design, performance, and testing	306
143.5	Exercises	306
143.6	Summary and further study	306
144	Race Conditions	307
144.1	The problem and the mental model	307
144.2	A small working program	307
144.3	Rules, mistakes, and diagnostics	307
144.4	Design, performance, and testing	308
144.5	Exercises	308
144.6	Summary and further study	308
145	Mutual Exclusion	309
145.1	The problem and the mental model	309
145.2	A small working program	309
145.3	Rules, mistakes, and diagnostics	309
145.4	Design, performance, and testing	310

145.5	Exercises	310
145.6	Summary and further study	310
146	Locks	311
146.1	The problem and the mental model	311
146.2	A small working program	311
146.3	Rules, mistakes, and diagnostics	311
146.4	Design, performance, and testing	312
146.5	Exercises	312
146.6	Summary and further study	312
147	Condition Variables	313
147.1	The problem and the mental model	313
147.2	A small working program	313
147.3	Rules, mistakes, and diagnostics	313
147.4	Design, performance, and testing	314
147.5	Exercises	314
147.6	Summary and further study	314
148	Futures and Asynchronous Work	315
148.1	The problem and the mental model	315
148.2	A small working program	315
148.3	Rules, mistakes, and diagnostics	315
148.4	Design, performance, and testing	316
148.5	Exercises	316
148.6	Summary and further study	316
149	Atomic Operations	317
149.1	The problem and the mental model	317
149.2	A small working program	317
149.3	Rules, mistakes, and diagnostics	317
149.4	Design, performance, and testing	318
149.5	Exercises	318
149.6	Summary and further study	318
150	The C++ Memory Model	319
150.1	The problem and the mental model	319
150.2	A small working program	319
150.3	Rules, mistakes, and diagnostics	319
150.4	Design, performance, and testing	320
150.5	Exercises	320
150.6	Summary and further study	320
151	Deadlocks and Livelocks	321
151.1	The problem and the mental model	321
151.2	A small working program	321
151.3	Rules, mistakes, and diagnostics	321
151.4	Design, performance, and testing	322
151.5	Exercises	322
151.6	Summary and further study	322
152	Thread-Safe Design	323
152.1	The problem and the mental model	323

152.2	A small working program	323
152.3	Rules, mistakes, and diagnostics	323
152.4	Design, performance, and testing	324
152.5	Exercises	324
152.6	Summary and further study	324
153	Parallel Algorithms	325
153.1	The problem and the mental model	325
153.2	A small working program	325
153.3	Rules, mistakes, and diagnostics	325
153.4	Design, performance, and testing	326
153.5	Exercises	326
153.6	Summary and further study	326
154	Task-Based Concurrency	327
154.1	The problem and the mental model	327
154.2	A small working program	327
154.3	Rules, mistakes, and diagnostics	327
154.4	Design, performance, and testing	328
154.5	Exercises	328
154.6	Summary and further study	328
XI	Performance and Systems	329
Part 11:	Performance and Systems	330
155	What Performance Means	331
155.1	The problem and the mental model	331
155.2	A small working program	331
155.3	Rules, mistakes, and diagnostics	331
155.4	Design, performance, and testing	332
155.5	Exercises	332
155.6	Summary and further study	332
156	Profiling Before Optimization	333
156.1	The problem and the mental model	333
156.2	A small working program	333
156.3	Rules, mistakes, and diagnostics	333
156.4	Design, performance, and testing	334
156.5	Exercises	334
156.6	Summary and further study	334
157	CPU, Cache, and Memory Locality	335
157.1	The problem and the mental model	335
157.2	A small working program	335
157.3	Rules, mistakes, and diagnostics	335
157.4	Design, performance, and testing	336
157.5	Exercises	336
157.6	Summary and further study	336
158	Allocation Costs	337
158.1	The problem and the mental model	337

158.2	A small working program	337
158.3	Rules, mistakes, and diagnostics	337
158.4	Design, performance, and testing	338
158.5	Exercises	338
158.6	Summary and further study	338
159	Data-Oriented Design	339
159.1	The problem and the mental model	339
159.2	A small working program	339
159.3	Rules, mistakes, and diagnostics	339
159.4	Design, performance, and testing	340
159.5	Exercises	340
159.6	Summary and further study	340
160	Inlining and Optimization	341
160.1	The problem and the mental model	341
160.2	A small working program	341
160.3	Rules, mistakes, and diagnostics	341
160.4	Design, performance, and testing	342
160.5	Exercises	342
160.6	Summary and further study	342
161	Compiler Optimization	343
161.1	The problem and the mental model	343
161.2	A small working program	343
161.3	Rules, mistakes, and diagnostics	343
161.4	Design, performance, and testing	344
161.5	Exercises	344
161.6	Summary and further study	344
162	Benchmark Design	345
162.1	The problem and the mental model	345
162.2	A small working program	345
162.3	Rules, mistakes, and diagnostics	345
162.4	Design, performance, and testing	346
162.5	Exercises	346
162.6	Summary and further study	346
163	Latency and Throughput	347
163.1	The problem and the mental model	347
163.2	A small working program	347
163.3	Rules, mistakes, and diagnostics	347
163.4	Design, performance, and testing	348
163.5	Exercises	348
163.6	Summary and further study	348
164	Performance Portability	349
164.1	The problem and the mental model	349
164.2	A small working program	349
164.3	Rules, mistakes, and diagnostics	349
164.4	Design, performance, and testing	350
164.5	Exercises	350
164.6	Summary and further study	350

165	Safe Optimization	351
165.1	The problem and the mental model	351
165.2	A small working program	351
165.3	Rules, mistakes, and diagnostics	351
165.4	Design, performance, and testing	352
165.5	Exercises	352
165.6	Summary and further study	352
166	When C++ Is an Appropriate Tool	353
166.1	The problem and the mental model	353
166.2	A small working program	353
166.3	Rules, mistakes, and diagnostics	353
166.4	Design, performance, and testing	354
166.5	Exercises	354
166.6	Summary and further study	354
167	When Another Language May Be Better	355
167.1	The problem and the mental model	355
167.2	A small working program	355
167.3	Rules, mistakes, and diagnostics	355
167.4	Design, performance, and testing	356
167.5	Exercises	356
167.6	Summary and further study	356
XII	Advanced Language and Design	357
Part 12:	Advanced Language and Design	358
168	Advanced Type-System Concepts	359
168.1	The problem and the mental model	359
168.2	A small working program	359
168.3	Rules, mistakes, and diagnostics	359
168.4	Design, performance, and testing	360
168.5	Exercises	360
168.6	Summary and further study	360
169	Perfect Forwarding	361
169.1	The problem and the mental model	361
169.2	A small working program	361
169.3	Rules, mistakes, and diagnostics	361
169.4	Design, performance, and testing	362
169.5	Exercises	362
169.6	Summary and further study	362
170	Customization and Generic Interfaces	363
170.1	The problem and the mental model	363
170.2	A small working program	363
170.3	Rules, mistakes, and diagnostics	363
170.4	Design, performance, and testing	364
170.5	Exercises	364
170.6	Summary and further study	364

171	Allocators and Memory Resources	365
171.1	The problem and the mental model	365
171.2	A small working program	365
171.3	Rules, mistakes, and diagnostics	365
171.4	Design, performance, and testing	366
171.5	Exercises	366
171.6	Summary and further study	366
172	Coroutines	367
172.1	The problem and the mental model	367
172.2	A small working program	367
172.3	Rules, mistakes, and diagnostics	367
172.4	Design, performance, and testing	368
172.5	Exercises	368
172.6	Summary and further study	368
173	Advanced Ranges	369
173.1	The problem and the mental model	369
173.2	A small working program	369
173.3	Rules, mistakes, and diagnostics	369
173.4	Design, performance, and testing	370
173.5	Exercises	370
173.6	Summary and further study	370
174	Metaprogramming	371
174.1	The problem and the mental model	371
174.2	A small working program	371
174.3	Rules, mistakes, and diagnostics	371
174.4	Design, performance, and testing	372
174.5	Exercises	372
174.6	Summary and further study	372
175	Type Erasure	373
175.1	The problem and the mental model	373
175.2	A small working program	373
175.3	Rules, mistakes, and diagnostics	373
175.4	Design, performance, and testing	374
175.5	Exercises	374
175.6	Summary and further study	374
176	Policy-Based Design	375
176.1	The problem and the mental model	375
176.2	A small working program	375
176.3	Rules, mistakes, and diagnostics	375
176.4	Design, performance, and testing	376
176.5	Exercises	376
176.6	Summary and further study	376
177	Architectural Patterns	377
177.1	The problem and the mental model	377
177.2	A small working program	377
177.3	Rules, mistakes, and diagnostics	377
177.4	Design, performance, and testing	378

177.5	Exercises	378
177.6	Summary and further study	378
178	Design Patterns in Modern C++	379
178.1	The problem and the mental model	379
178.2	A small working program	379
178.3	Rules, mistakes, and diagnostics	379
178.4	Design, performance, and testing	380
178.5	Exercises	380
178.6	Summary and further study	380
179	Domain Modeling	381
179.1	The problem and the mental model	381
179.2	A small working program	381
179.3	Rules, mistakes, and diagnostics	381
179.4	Design, performance, and testing	382
179.5	Exercises	382
179.6	Summary and further study	382
180	Stable Interfaces	383
180.1	The problem and the mental model	383
180.2	A small working program	383
180.3	Rules, mistakes, and diagnostics	383
180.4	Design, performance, and testing	384
180.5	Exercises	384
180.6	Summary and further study	384
181	Large-Codebase Architecture	385
181.1	The problem and the mental model	385
181.2	A small working program	385
181.3	Rules, mistakes, and diagnostics	385
181.4	Design, performance, and testing	386
181.5	Exercises	386
181.6	Summary and further study	386
182	Legacy-Code Modernization	387
182.1	The problem and the mental model	387
182.2	A small working program	387
182.3	Rules, mistakes, and diagnostics	387
182.4	Design, performance, and testing	388
182.5	Exercises	388
182.6	Summary and further study	388
	Project index	389
	Glossary	390
	Bibliographic note	391
	Index	393

PART I

Computers and Programming

Part 1: Computers and Programming

The machine model, translation pipeline, tools, and the first complete program.

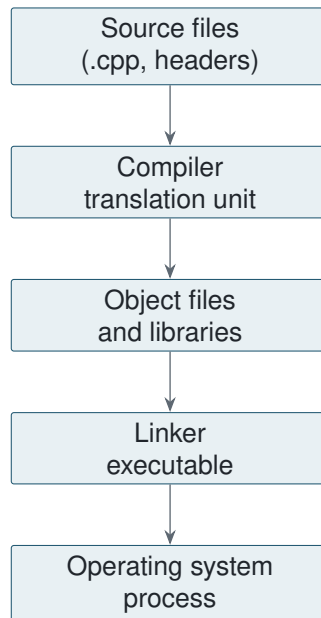


Figure 1: A simplified translation pipeline. The exact tools and intermediate files vary by platform, but the boundaries are useful for debugging.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

What a Computer Program Is

Learning objectives

- Explain the central problem addressed by What a Computer Program Is and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the orientation in the front matter, plus the vocabulary introduced in Part 1.

Key terminology: What a Computer Program Is; contract; representation; lifetime; invariant; diagnostic; trade-off.

1.1 The problem and the mental model

A program is a description of a computation: inputs are transformed by rules into outputs or effects. The chapter distinguishes source text from the behavior a machine eventually performs and introduces the idea of an interface between a program and its environment.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

1.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch001.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 1: What a Computer Program Is
#include <iostream>

int main() {
    std::cout << "Chapter 1: What a Computer Program Is" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

1.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

1.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses what a computer program is and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

1.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

1.6 Summary and further study

What a Computer Program Is is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Hardware, Software, and Operating Systems

Learning objectives

- Explain the central problem addressed by Hardware, Software, and Operating Systems and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 1.

Key terminology: Hardware, Software, and Operating Systems; contract; representation; lifetime; invariant; diagnostic; trade-off.

2.1 The problem and the mental model

Programs run within layers. Processors execute instructions, memory stores state, devices expose effects, and operating systems mediate resources. The useful model is layered rather than a claim that every computer has one physical layout.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

2.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch002.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 2: Hardware, Software, and Operating Systems
#include <iostream>

int main() {
    std::cout << "Chapter 2: Hardware, Software, and Operating Systems" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

2.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

2.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses hardware, software, and operating systems and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

2.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

2.6 Summary and further study

Hardware, Software, and Operating Systems is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Source Code, Machine Code, and Executable Files

Learning objectives

- Explain the central problem addressed by Source Code, Machine Code, and Executable Files and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 1.

Key terminology: Source Code, Machine Code, and Executable Files; contract; representation; lifetime; invariant; diagnostic; trade-off.

3.1 The problem and the mental model

Human-readable source is translated into target-specific object code and eventually an executable image. The chapter explains why the same source can produce different binaries without making portability mysterious.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

3.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch003.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 3: Source Code, Machine Code, and Executable Files
#include <iostream>

int main() {
    std::cout << "Chapter 3: Source Code, Machine Code, and Executable Files" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

3.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

3.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses source code, machine code, and executable files and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

3.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

3.6 Summary and further study

Source Code, Machine Code, and Executable Files is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Compilers, Linkers, Libraries, and Build Processes

Learning objectives

- Explain the central problem addressed by Compilers, Linkers, Libraries, and Build Processes and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 1.

Key terminology: Compilers, Linkers, Libraries, and Build Processes; contract; representation; lifetime; invariant; diagnostic; trade-off.

4.1 The problem and the mental model

A compiler checks and translates a translation unit; a linker resolves references among object files and libraries. Build systems make those steps repeatable and avoid needless work.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

4.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch004.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 4: Compilers, Linkers, Libraries, and Build Processes
#include <iostream>

int main() {
    std::cout << "Chapter 4: Compilers, Linkers, Libraries, and Build Processes" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

4.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

4.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses compilers, linkers, libraries, and build processes and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

4.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

4.6 Summary and further study

Compilers, Linkers, Libraries, and Build Processes is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Installing a C++ Development Environment

Learning objectives

- Explain the central problem addressed by Installing a C++ Development Environment and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 1.

Key terminology: Installing a C++ Development Environment; contract; representation; lifetime; invariant; diagnostic; trade-off.

5.1 The problem and the mental model

A usable environment contains a compiler, standard library, debugger, editor, and build tools. Installation is treated as a reproducible configuration rather than a sequence of unexplained clicks.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

5.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch005.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 5: Installing a C++ Development Environment
#include <iostream>

int main() {
    std::cout << "Chapter 5: Installing a C++ Development Environment" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

5.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

5.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses installing a c++ development environment and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

5.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

5.6 Summary and further study

Installing a C++ Development Environment is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Terminals, Editors, IDEs, and File Systems

Learning objectives

- Explain the central problem addressed by Terminals, Editors, IDEs, and File Systems and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 1.

Key terminology: Terminals, Editors, IDEs, and File Systems; contract; representation; lifetime; invariant; diagnostic; trade-off.

6.1 The problem and the mental model

The terminal is a text interface to processes, not a programming language. Editors and IDEs add navigation and diagnostics, while the file system gives source files durable names and locations.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

6.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch006.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 6: Terminals, Editors, IDEs, and File Systems
#include <iostream>

int main() {
    std::cout << "Chapter 6: Terminals, Editors, IDEs, and File Systems" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

6.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

6.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses terminals, editors, ides, and file systems and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

6.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

6.6 Summary and further study

Terminals, Editors, IDEs, and File Systems is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Writing, Compiling, and Running the First Program

Learning objectives

- Explain the central problem addressed by Writing, Compiling, and Running the First Program and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 1.

Key terminology: Writing, Compiling, and Running the First Program; contract; representation; lifetime; invariant; diagnostic; trade-off.

7.1 The problem and the mental model

The first program makes the translation pipeline visible: create a source file, invoke the compiler, inspect the executable, run it, and observe standard output and the exit status.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

7.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch007.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 7: Writing, Compiling, and Running the First Program
#include <iostream>

int main() {
    std::cout << "Hello, modern C++!\n";
    return 0;
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

7.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

7.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses writing, compiling, and running the first program and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

7.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

7.6 Summary and further study

Writing, Compiling, and Running the First Program is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Reading Compiler Diagnostics

Learning objectives

- Explain the central problem addressed by Reading Compiler Diagnostics and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 1.

Key terminology: Reading Compiler Diagnostics; contract; representation; lifetime; invariant; diagnostic; trade-off.

8.1 The problem and the mental model

Diagnostics are evidence about a failed translation, not personal judgement. The chapter teaches readers to locate the first useful message, reduce the problem, and distinguish syntax, type, and linker failures.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

8.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch008.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 8: Reading Compiler Diagnostics
#include <iostream>

int main() {
    std::cout << "Chapter 8: Reading Compiler Diagnostics" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

8.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

8.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses reading compiler diagnostics and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

8.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

8.6 Summary and further study

Reading Compiler Diagnostics is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART II

The Foundations of C++

Part 2: The Foundations of C++

The core language vocabulary needed to read, write, and reason about ordinary programs.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Program Structure

Learning objectives

- Explain the central problem addressed by Program Structure and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Program Structure; contract; representation; lifetime; invariant; diagnostic; trade-off.

9.1 The problem and the mental model

C++ programs are made from declarations, definitions, functions, namespaces, and statements. Structure determines what names mean and what the implementation is allowed to do.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

9.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch009.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 9: Program Structure
#include <iostream>

int main() {
    std::cout << "Chapter 9: Program Structure" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

9.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

9.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses program structure and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

9.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

9.6 Summary and further study

Program Structure is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Statements, Expressions, and Blocks

Learning objectives

- Explain the central problem addressed by Statements, Expressions, and Blocks and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Statements, Expressions, and Blocks; contract; representation; lifetime; invariant; diagnostic; trade-off.

10.1 The problem and the mental model

An expression computes a value or effect; a statement organizes execution; a block introduces a nested region. Understanding the distinction makes control flow and diagnostics easier to read.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

10.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch010.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 10: Statements, Expressions, and Blocks
#include <iostream>

int main() {
    std::cout << "Chapter 10: Statements, Expressions, and Blocks" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

10.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

10.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses statements, expressions, and blocks and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

10.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

10.6 Summary and further study

Statements, Expressions, and Blocks is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Variables and Initialization

Learning objectives

- Explain the central problem addressed by Variables and Initialization and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Variables and Initialization; contract; representation; lifetime; invariant; diagnostic; trade-off.

11.1 The problem and the mental model

A variable gives an object a name and a type. Initialization creates its initial state, while assignment changes an already existing object; confusing the two causes subtle bugs.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

11.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch011.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 11: Variables and Initialization
#include <iostream>

int main() {
    const int items = 3;
    const double price = 4.50;
    std::cout << items * price << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

11.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

11.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses variables and initialization and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

11.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

11.6 Summary and further study

Variables and Initialization is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Fundamental Types

Learning objectives

- Explain the central problem addressed by Fundamental Types and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Fundamental Types; contract; representation; lifetime; invariant; diagnostic; trade-off.

12.1 The problem and the mental model

Fundamental types describe categories such as integers, floating-point values, characters, booleans, and void. Their sizes and representations have implementation-defined aspects, so portable code relies on guarantees rather than guesses.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

12.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch012.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 12: Fundamental Types
#include <iostream>

int main() {
    std::cout << "Chapter 12: Fundamental Types" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

12.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

12.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses fundamental types and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

12.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

12.6 Summary and further study

Fundamental Types is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Literals

Learning objectives

- Explain the central problem addressed by Literals and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Literals; contract; representation; lifetime; invariant; diagnostic; trade-off.

13.1 The problem and the mental model

Literals are source-level notation for values: numeric, character, string, boolean, and null-pointer literals. Suffixes and prefixes can change type and meaning.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

13.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch013.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 13: Literals
#include <iostream>

int main() {
    std::cout << "Chapter 13: Literals" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

13.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

13.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses literals and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

13.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

13.6 Summary and further study

Literals is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Arithmetic and Operators

Learning objectives

- Explain the central problem addressed by Arithmetic and Operators and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Arithmetic and Operators; contract; representation; lifetime; invariant; diagnostic; trade-off.

14.1 The problem and the mental model

Operators are syntax with specified operands, result types, sequencing, and possible conversions. Arithmetic becomes safer when ranges, overflow, and integer division are made explicit.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

14.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch014.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 14: Arithmetic and Operators
#include <iostream>

int main() {
    std::cout << "Chapter 14: Arithmetic and Operators" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

14.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

14.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses arithmetic and operators and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

14.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

14.6 Summary and further study

Arithmetic and Operators is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Comparisons and Boolean Logic

Learning objectives

- Explain the central problem addressed by Comparisons and Boolean Logic and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Comparisons and Boolean Logic; contract; representation; lifetime; invariant; diagnostic; trade-off.

15.1 The problem and the mental model

Conditions combine comparisons and logical operators. Short-circuit evaluation is useful, but only when readers understand which operands are evaluated and when.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

15.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch015.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 15: Comparisons and Boolean Logic
#include <iostream>

int main() {
    std::cout << "Chapter 15: Comparisons and Boolean Logic" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

15.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

15.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses comparisons and boolean logic and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

15.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

15.6 Summary and further study

Comparisons and Boolean Logic is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Input and Output

Learning objectives

- Explain the central problem addressed by Input and Output and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Input and Output; contract; representation; lifetime; invariant; diagnostic; trade-off.

16.1 The problem and the mental model

Streams model formatted interaction with sources and sinks. The important engineering questions are parsing, failure states, buffering, and whether interactive input belongs in the domain logic.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

16.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch016.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 16: Input and Output
#include <iostream>

int main() {
    std::cout << "Chapter 16: Input and Output" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

16.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

16.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses input and output and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

16.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

16.6 Summary and further study

Input and Output is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Selection with if and switch

Learning objectives

- Explain the central problem addressed by Selection with if and switch and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Selection with if and switch; contract; representation; lifetime; invariant; diagnostic; trade-off.

17.1 The problem and the mental model

Selection chooses behavior based on a condition or a discrete value. Good branches make invalid states visible and keep policy separate from mechanism.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

17.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch017.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 17: Selection with if and switch
#include <iostream>

int main() {
    const int score = 72;
    if (score >= 60) {
        std::cout << "pass\n";
    } else {
        std::cout << "retry\n";
    }
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

17.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

17.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses selection with `if` and `switch` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

17.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

17.6 Summary and further study

Selection with `if` and `switch` is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Repetition with for, while, and do-while

Learning objectives

- Explain the central problem addressed by Repetition with for, while, and do-while and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Repetition with for, while, and do-while; contract; representation; lifetime; invariant; diagnostic; trade-off.

18.1 The problem and the mental model

Loops express repeated work and an exit condition. Correct loop reasoning names the changing state, the termination condition, and the progress that makes termination plausible.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

18.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch018.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 18: Repetition with for, while, and do-while
#include <iostream>

int main() {
    int total = 0;
    for (int value = 1; value <= 5; ++value) {
        total += value;
    }
    std::cout << total << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

18.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

18.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses repetition with `for`, `while`, and `do-while` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

18.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

18.6 Summary and further study

Repetition with `for`, `while`, and `do-while` is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Scope, Lifetime, and Name Visibility

Learning objectives

- Explain the central problem addressed by Scope, Lifetime, and Name Visibility and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Scope, Lifetime, and Name Visibility; contract; representation; lifetime; invariant; diagnostic; trade-off.

19.1 The problem and the mental model

Scope answers where a name can be used; lifetime answers when an object exists. They often align for local variables but are not the same concept.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

19.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch019.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 19: Scope, Lifetime, and Name Visibility
#include <iostream>

int main() {
    std::cout << "Chapter 19: Scope, Lifetime, and Name Visibility" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

19.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

19.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses scope, lifetime, and name visibility and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

19.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

19.6 Summary and further study

Scope, Lifetime, and Name Visibility is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Functions

Learning objectives

- Explain the central problem addressed by Functions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Functions; contract; representation; lifetime; invariant; diagnostic; trade-off.

20.1 The problem and the mental model

Functions name reusable behavior and establish a boundary for inputs, outputs, side effects, and tests. Small functions are useful when their boundaries reflect meaningful responsibilities.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

20.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch020.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 20: Functions
#include <iostream>

int square(int value) {
    return value * value;
}

int main() {
    std::cout << square(7) << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

20.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

20.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses functions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

20.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

20.6 Summary and further study

Functions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Function Parameters and Return Values

Learning objectives

- Explain the central problem addressed by Function Parameters and Return Values and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Function Parameters and Return Values; contract; representation; lifetime; invariant; diagnostic; trade-off.

21.1 The problem and the mental model

Parameters and return values form a function’s value-level interface. Passing by value, reference, pointer, or view communicates different ownership and lifetime expectations.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

21.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch021.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 21: Function Parameters and Return Values
#include <iostream>

int main() {
    std::cout << "Chapter 21: Function Parameters and Return Values" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

21.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

21.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses function parameters and return values and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

21.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

21.6 Summary and further study

Function Parameters and Return Values is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Overloading and Default Arguments

Learning objectives

- Explain the central problem addressed by Overloading and Default Arguments and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Overloading and Default Arguments; contract; representation; lifetime; invariant; diagnostic; trade-off.

22.1 The problem and the mental model

Overloading lets one name denote several signatures, while default arguments fill omitted values. Both features can improve interfaces but can also create ambiguity and surprising calls.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

22.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch022.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 22: Overloading and Default Arguments
#include <iostream>

int main() {
    std::cout << "Chapter 22: Overloading and Default Arguments" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

22.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

22.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses overloading and default arguments and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

22.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

22.6 Summary and further study

Overloading and Default Arguments is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Type Deduction with auto

Learning objectives

- Explain the central problem addressed by Type Deduction with auto and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Type Deduction with auto; contract; representation; lifetime; invariant; diagnostic; trade-off.

23.1 The problem and the mental model

auto asks the compiler to deduce a type from an initializer. It removes repetition without removing type checking, but reference, const, and value-category rules still matter.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

23.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch023.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 23: Type Deduction with auto
#include <iostream>

int main() {
    std::cout << "Chapter 23: Type Deduction with auto" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

23.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

23.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses type deduction with `auto` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

23.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

23.6 Summary and further study

Type Deduction with `auto` is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Constants and constexpr

Learning objectives

- Explain the central problem addressed by Constants and constexpr and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Constants and constexpr; contract; representation; lifetime; invariant; diagnostic; trade-off.

24.1 The problem and the mental model

const prevents mutation through a particular name; constexpr makes constant evaluation possible when the rules permit it. Compile-time computation is a guarantee about evaluation, not a promise that every function runs at compile time.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

24.2 A small working program

The complete standalone program for this chapter is stored at examples/chapters/ch024.cpp and is compiled by the verification script.

```
// Modern C++ - Chapter 24: Constants and constexpr
#include <iostream>

int main() {
    std::cout << "Chapter 24: Constants and constexpr" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

24.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

24.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses constants and `constexpr` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

24.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

24.6 Summary and further study

Constants and `constexpr` is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Enumerations

Learning objectives

- Explain the central problem addressed by Enumerations and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Enumerations; contract; representation; lifetime; invariant; diagnostic; trade-off.

25.1 The problem and the mental model

Enumerations represent a closed set of named values. Scoped enumerations make conversions and ownership of names more explicit than unscoped alternatives.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

25.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch025.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 25: Enumerations
#include <iostream>

int main() {
    std::cout << "Chapter 25: Enumerations" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

25.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

25.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses enumerations and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

25.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

25.6 Summary and further study

Enumerations is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Namespaces

Learning objectives

- Explain the central problem addressed by Namespaces and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 2.

Key terminology: Namespaces; contract; representation; lifetime; invariant; diagnostic; trade-off.

26.1 The problem and the mental model

Namespaces control name collisions and communicate ownership. Qualification is often clearer than broad using-directives, particularly in headers and public interfaces.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

26.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch026.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 26: Namespaces
#include <iostream>

int main() {
    std::cout << "Chapter 26: Namespaces" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

26.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

26.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses namespaces and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

26.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

26.6 Summary and further study

Namespaces is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART III

Data and Memory

Part 3: Data and Memory

Representation, text, containers, pointers, ownership, and object lifetime without folklore.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Binary Representation

Learning objectives

- Explain the central problem addressed by Binary Representation and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Binary Representation; contract; representation; lifetime; invariant; diagnostic; trade-off.

27.1 The problem and the mental model

Bits are a model for representing information, not a license to assume a particular machine layout. The chapter introduces bases, bytes, object representations, and the limits of representation-based reasoning.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

27.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch027.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 27: Binary Representation
#include <iostream>

int main() {
    std::cout << "Chapter 27: Binary Representation" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

27.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

27.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses binary representation and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

27.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

27.6 Summary and further study

Binary Representation is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Integers, Floating-Point Values, and Precision

Learning objectives

- Explain the central problem addressed by Integers, Floating-Point Values, and Precision and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Integers, Floating-Point Values, and Precision; contract; representation; lifetime; invariant; diagnostic; trade-off.

28.1 The problem and the mental model

Integer arithmetic and floating-point arithmetic have different error models. Range, rounding, comparison, and conversion choices should follow the problem rather than intuition from decimal notation.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

28.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch028.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 28: Integers, Floating-Point Values, and Precision
#include <iostream>

int main() {
    std::cout << "Chapter 28: Integers, Floating-Point Values, and Precision" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

28.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

28.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses integers, floating-point values, and precision and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

28.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

28.6 Summary and further study

Integers, Floating-Point Values, and Precision is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Characters, Text, Encoding, and Unicode

Learning objectives

- Explain the central problem addressed by Characters, Text, Encoding, and Unicode and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Characters, Text, Encoding, and Unicode; contract; representation; lifetime; invariant; diagnostic; trade-off.

29.1 The problem and the mental model

A character, a code point, a code unit, and a user-perceived grapheme are distinct ideas. `std::string` stores bytes; text correctness requires an explicit encoding policy.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

29.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch029.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 29: Characters, Text, Encoding, and Unicode
#include <iostream>

int main() {
    std::cout << "Chapter 29: Characters, Text, Encoding, and Unicode" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

29.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

29.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses characters, text, encoding, and unicode and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

29.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

29.6 Summary and further study

Characters, Text, Encoding, and Unicode is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Arrays

Learning objectives

- Explain the central problem addressed by Arrays and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Arrays; contract; representation; lifetime; invariant; diagnostic; trade-off.

30.1 The problem and the mental model

Built-in arrays provide contiguous elements and fixed extent, but decay and bounds mistakes make interfaces fragile. The chapter explains what the type really is before introducing safer views.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

30.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch030.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 30: Arrays
#include <iostream>

int main() {
    std::cout << "Chapter 30: Arrays" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

30.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

30.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses arrays and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

30.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

30.6 Summary and further study

Arrays is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

std::array

Learning objectives

- Explain the central problem addressed by std::array and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: std::array; contract; representation; lifetime; invariant; diagnostic; trade-off.

31.1 The problem and the mental model

std::array gives a fixed-size sequence with value semantics, standard-library interoperability, and a size that is part of the type. It is often a better default than a built-in array.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

31.2 A small working program

The complete standalone program for this chapter is stored at examples/chapters/ch031.cpp and is compiled by the verification script.

```
// Modern C++ - Chapter 31: std::array
#include <iostream>

int main() {
    std::cout << "Chapter 31: std::array" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

31.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

31.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses `std::array` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

31.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

31.6 Summary and further study

`std::array` is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

std::vector

Learning objectives

- Explain the central problem addressed by std::vector and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: std::vector; contract; representation; lifetime; invariant; diagnostic; trade-off.

32.1 The problem and the mental model

std::vector owns a contiguous dynamic sequence. Size, capacity, iterator invalidation, and reallocation determine which references remain valid after a mutation.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

32.2 A small working program

The complete standalone program for this chapter is stored at examples/chapters/ch032.cpp and is compiled by the verification script.

```
// Modern C++ - Chapter 32: std::vector
#include <iostream>
#include <vector>

int main() {
    const std::vector<int> values{2, 4, 6, 8};
    for (const int value : values) {
        std::cout << value << " ";
    }
    std::cout << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

32.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

32.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses `std::vector` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

32.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

32.6 Summary and further study

`std::vector` is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

std::string and std::string_view

Learning objectives

- Explain the central problem addressed by std::string and std::string_view and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: std::string and std::string_view; contract; representation; lifetime; invariant; diagnostic; trade-off.

33.1 The problem and the mental model

std::string owns text storage; std::string_view borrows a range of characters. A view is efficient only when its source outlives every use of the view.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

33.2 A small working program

The complete standalone program for this chapter is stored at examples/chapters/ch033.cpp and is compiled by the verification script.

```
// Modern C++ - Chapter 33: std::string and std::string_view
#include <iostream>
#include <string>

int main() {
    const std::string language = "C++";
    std::cout << language.size() << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

33.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

33.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses `std::string` and `std::string_view` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

33.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

33.6 Summary and further study

`std::string` and `std::string_view` is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

References

Learning objectives

- Explain the central problem addressed by References and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: References; contract; representation; lifetime; invariant; diagnostic; trade-off.

34.1 The problem and the mental model

A reference is an alias with binding rules, not a reseatable pointer and not an ownership token. References make non-null borrowing expressive but do not extend lifetime automatically.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

34.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch034.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 34: References
#include <iostream>

int main() {
    std::cout << "Chapter 34: References" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

34.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

34.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses references and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

34.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

34.6 Summary and further study

References is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Pointers

Learning objectives

- Explain the central problem addressed by Pointers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Pointers; contract; representation; lifetime; invariant; diagnostic; trade-off.

35.1 The problem and the mental model

A pointer is an object that can hold an address or a null value. It can represent optional access or navigation; it becomes ownership only through an explicit protocol.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

35.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch035.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 35: Pointers
#include <iostream>

int main() {
    int value = 42;
    int* pointer = &value;
    if (pointer != nullptr) {
        std::cout << *pointer << "\n";
    }
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

35.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

35.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses pointers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

35.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

35.6 Summary and further study

Pointers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Null Pointers

Learning objectives

- Explain the central problem addressed by Null Pointers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Null Pointers; contract; representation; lifetime; invariant; diagnostic; trade-off.

36.1 The problem and the mental model

A null pointer represents no pointed-to object. Comparing, testing, and avoiding dereference are simple rules, but nullable interfaces should be used only when absence is meaningful.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

36.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch036.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 36: Null Pointers
#include <iostream>

int main() {
    std::cout << "Chapter 36: Null Pointers" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

36.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

36.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses null pointers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

36.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

36.6 Summary and further study

Null Pointers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Dynamic Storage

Learning objectives

- Explain the central problem addressed by Dynamic Storage and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Dynamic Storage; contract; representation; lifetime; invariant; diagnostic; trade-off.

37.1 The problem and the mental model

Dynamic storage supports objects whose size or lifetime is not conveniently tied to a local scope. Modern C++ hides raw allocation behind containers and resource-owning types whenever possible.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

37.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch037.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 37: Dynamic Storage
#include <iostream>

int main() {
    std::cout << "Chapter 37: Dynamic Storage" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

37.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

37.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses dynamic storage and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

37.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

37.6 Summary and further study

Dynamic Storage is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Stack and Free-Store Concepts

Learning objectives

- Explain the central problem addressed by Stack and Free-Store Concepts and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Stack and Free-Store Concepts; contract; representation; lifetime; invariant; diagnostic; trade-off.

38.1 The problem and the mental model

Stack and free-store are useful implementation-level terms, not universal standard categories. The language specifies storage duration and lifetime; implementations choose how those are realized.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

38.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch038.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 38: Stack and Free-Store Concepts
#include <iostream>

int main() {
    std::cout << "Chapter 38: Stack and Free-Store Concepts" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

38.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

38.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses stack and free-store concepts and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

38.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

38.6 Summary and further study

Stack and Free-Store Concepts is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Object Lifetime

Learning objectives

- Explain the central problem addressed by Object Lifetime and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Object Lifetime; contract; representation; lifetime; invariant; diagnostic; trade-off.

39.1 The problem and the mental model

An object has a lifetime with a beginning and an end. Using storage outside that lifetime, or with the wrong type, is a correctness error even if the bytes look unchanged.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

39.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch039.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 39: Object Lifetime
#include <iostream>

int main() {
    std::cout << "Chapter 39: Object Lifetime" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

39.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

39.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses object lifetime and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

39.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

39.6 Summary and further study

Object Lifetime is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Resource Ownership

Learning objectives

- Explain the central problem addressed by Resource Ownership and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Resource Ownership; contract; representation; lifetime; invariant; diagnostic; trade-off.

40.1 The problem and the mental model

Files, locks, sockets, and memory all require a release protocol. Ownership answers who is responsible, when responsibility transfers, and how failure preserves the protocol.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

40.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch040.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 40: Resource Ownership
#include <iostream>

int main() {
    std::cout << "Chapter 40: Resource Ownership" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

40.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

40.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses resource ownership and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

40.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

40.6 Summary and further study

Resource Ownership is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Smart Pointers

Learning objectives

- Explain the central problem addressed by Smart Pointers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Smart Pointers; contract; representation; lifetime; invariant; diagnostic; trade-off.

41.1 The problem and the mental model

`unique_ptr` expresses exclusive ownership, `shared_ptr` expresses shared ownership, and `weak_ptr` observes shared ownership without keeping it alive. The choice is a design statement, not a performance superstition.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

41.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch041.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 41: Smart Pointers
#include <iostream>
#include <memory>

int main() {
    auto value = std::make_unique<int>(42);
    std::cout << *value << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

41.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

41.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses smart pointers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

41.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

41.6 Summary and further study

Smart Pointers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

RAII

Learning objectives

- Explain the central problem addressed by RAII and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: RAII; contract; representation; lifetime; invariant; diagnostic; trade-off.

42.1 The problem and the mental model

Resource acquisition is initialization binds release to deterministic destruction. RAII makes early returns and exceptions ordinary control-flow paths rather than leaks to be remembered manually.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

42.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch042.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 42: RAII
#include <iostream>
#include <string>
#include <utility>

class Message {
public:
    explicit Message(std::string text) : text_(std::move(text)) {
        std::cout << "acquired\n";
    }
    ~Message() { std::cout << "released\n"; }
private:
    std::string text_;
};

int main() {
    Message message{"RAII"};
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

42.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

42.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses `raii` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

42.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

42.6 Summary and further study

RAII is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Copy and Move Semantics

Learning objectives

- Explain the central problem addressed by Copy and Move Semantics and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Copy and Move Semantics; contract; representation; lifetime; invariant; diagnostic; trade-off.

43.1 The problem and the mental model

Copying duplicates a value or resource representation; moving transfers or cheaply reuses resources. A type's special members should preserve its invariants and ownership rules.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

43.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch043.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 43: Copy and Move Semantics
#include <iostream>
#include <string>
#include <utility>

int main() {
    std::string original = "resource";
    std::string moved = std::move(original);
    std::cout << moved << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

43.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

43.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses copy and move semantics and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

43.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

43.6 Summary and further study

Copy and Move Semantics is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Value Categories

Learning objectives

- Explain the central problem addressed by Value Categories and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Value Categories; contract; representation; lifetime; invariant; diagnostic; trade-off.

44.1 The problem and the mental model

Expressions have categories that affect binding, overload resolution, and moving. The useful beginner model is lvalue as an identifiable object and rvalue as a temporary or expiring value; the formal model is more precise.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

44.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch044.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 44: Value Categories
#include <iostream>

int main() {
    std::cout << "Chapter 44: Value Categories" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

44.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

44.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses value categories and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

44.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

44.6 Summary and further study

Value Categories is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Undefined, Unspecified, and Implementation-Defined Behavior

Learning objectives

- Explain the central problem addressed by Undefined, Unspecified, and Implementation-Defined Behavior and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 3.

Key terminology: Undefined, Unspecified, and Implementation-Defined Behavior; contract; representation; lifetime; invariant; diagnostic; trade-off.

45.1 The problem and the mental model

These labels describe different degrees of freedom in the standard. Correct engineering avoids undefined behavior, documents implementation choices, and does not mistake one observed result for a guarantee.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

45.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch045.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 45: Undefined, Unspecified, and Implementation-Defined Behavior
#include <iostream>

int main() {
    std::cout << "Chapter 45: Undefined, Unspecified, and Implementation-Defined Behavior" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

45.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

45.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses undefined, unspecified, and implementation-defined behavior and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

45.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

45.6 Summary and further study

Undefined, Unspecified, and Implementation-Defined Behavior is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART IV

User-Defined Types

Part 4: User-Defined Types

Classes, invariants, polymorphism, and a wider view of C++ design than object orientation alone.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Structures

Learning objectives

- Explain the central problem addressed by Structures and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Structures; contract; representation; lifetime; invariant; diagnostic; trade-off.

46.1 The problem and the mental model

A struct groups related data and can provide behavior. In C++, struct and class differ primarily in default access, not in expressive power.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

46.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch046.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 46: Structures
#include <iostream>

int main() {
    std::cout << "Chapter 46: Structures" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

46.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

46.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses structures and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

46.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

46.6 Summary and further study

Structures is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Classes and Objects

Learning objectives

- Explain the central problem addressed by Classes and Objects and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Classes and Objects; contract; representation; lifetime; invariant; diagnostic; trade-off.

47.1 The problem and the mental model

A class defines a type; an object is an instance with state and lifetime. A good type makes valid states easy to construct and invalid states difficult to represent.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

47.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch047.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 47: Classes and Objects
#include <iostream>

class Counter {
public:
    void increment() { ++value_; }
    int value() const { return value_; }
private:
    int value_{0};
};

int main() {
    Counter counter;
    counter.increment();
    std::cout << counter.value() << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

47.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

47.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses classes and objects and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

47.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

47.6 Summary and further study

Classes and Objects is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Access Control

Learning objectives

- Explain the central problem addressed by Access Control and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Access Control; contract; representation; lifetime; invariant; diagnostic; trade-off.

48.1 The problem and the mental model

public, protected, and private establish compile-time access boundaries. Encapsulation is not secrecy; it is control over which operations can preserve an abstraction's rules.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

48.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch048.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 48: Access Control
#include <iostream>

int main() {
    std::cout << "Chapter 48: Access Control" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

48.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

48.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses access control and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

48.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

48.6 Summary and further study

Access Control is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Constructors

Learning objectives

- Explain the central problem addressed by Constructors and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Constructors; contract; representation; lifetime; invariant; diagnostic; trade-off.

49.1 The problem and the mental model

Constructors establish initial invariants. Member-initializer lists, delegating constructors, and explicit conversions make construction predictable and resistant to partial initialization.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

49.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch049.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 49: Constructors
#include <iostream>

int main() {
    std::cout << "Chapter 49: Constructors" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

49.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

49.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses constructors and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

49.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

49.6 Summary and further study

Constructors is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Destructors

Learning objectives

- Explain the central problem addressed by Destructors and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Destructors; contract; representation; lifetime; invariant; diagnostic; trade-off.

50.1 The problem and the mental model

Destruction releases owned resources and ends an object's lifetime. Destructors should be reliable, bounded in responsibility, and normally non-throwing.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

50.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch050.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 50: Destructors
#include <iostream>

int main() {
    std::cout << "Chapter 50: Destructors" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

50.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

50.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses destructors and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

50.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

50.6 Summary and further study

Destructors is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Invariants

Learning objectives

- Explain the central problem addressed by Invariants and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Invariants; contract; representation; lifetime; invariant; diagnostic; trade-off.

51.1 The problem and the mental model

An invariant is a condition that must hold at defined boundaries. Naming invariants turns informal assumptions into reviewable design and test obligations.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

51.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch051.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 51: Invariants
#include <iostream>

int main() {
    std::cout << "Chapter 51: Invariants" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

51.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

51.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses invariants and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

51.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

51.6 Summary and further study

Invariants is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Member Functions

Learning objectives

- Explain the central problem addressed by Member Functions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Member Functions; contract; representation; lifetime; invariant; diagnostic; trade-off.

52.1 The problem and the mental model

Member functions operate with an implicit object parameter. Their constness, ref-qualification, and exception behavior are part of the type's interface.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

52.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch052.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 52: Member Functions
#include <iostream>

int main() {
    std::cout << "Chapter 52: Member Functions" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

52.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

52.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses member functions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

52.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

52.6 Summary and further study

Member Functions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

const Correctness

Learning objectives

- Explain the central problem addressed by const Correctness and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: const Correctness; contract; representation; lifetime; invariant; diagnostic; trade-off.

53.1 The problem and the mental model

const communicates non-mutation through a particular access path. It is a useful local guarantee, but it does not make pointed-to objects immutable or make concurrent access safe.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

53.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch053.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 53: const Correctness
#include <iostream>

int main() {
    std::cout << "Chapter 53: const Correctness" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

53.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

53.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses const correctness and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

53.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

53.6 Summary and further study

const Correctness is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Static Members

Learning objectives

- Explain the central problem addressed by Static Members and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Static Members; contract; representation; lifetime; invariant; diagnostic; trade-off.

54.1 The problem and the mental model

Static data and functions belong to the class rather than one object. They can represent shared policy or state, but global lifetime and synchronization costs should be explicit.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

54.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch054.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 54: Static Members
#include <iostream>

int main() {
    std::cout << "Chapter 54: Static Members" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

54.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

54.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses static members and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

54.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

54.6 Summary and further study

Static Members is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Operator Overloading

Learning objectives

- Explain the central problem addressed by Operator Overloading and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Operator Overloading; contract; representation; lifetime; invariant; diagnostic; trade-off.

55.1 The problem and the mental model

Overloaded operators can make domain values readable when they preserve familiar meaning. Operators should not disguise expensive effects or surprising state changes.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

55.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch055.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 55: Operator Overloading
#include <iostream>

int main() {
    std::cout << "Chapter 55: Operator Overloading" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

55.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

55.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses operator overloading and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

55.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

55.6 Summary and further study

Operator Overloading is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Composition

Learning objectives

- Explain the central problem addressed by Composition and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Composition; contract; representation; lifetime; invariant; diagnostic; trade-off.

56.1 The problem and the mental model

Composition builds a type from collaborating parts. It usually makes dependencies and replacement boundaries clearer than a deep inheritance hierarchy.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

56.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch056.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 56: Composition
#include <iostream>

int main() {
    std::cout << "Chapter 56: Composition" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

56.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

56.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses composition and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

56.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

56.6 Summary and further study

Composition is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Inheritance

Learning objectives

- Explain the central problem addressed by Inheritance and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Inheritance; contract; representation; lifetime; invariant; diagnostic; trade-off.

57.1 The problem and the mental model

Inheritance expresses a relationship between a derived type and a base interface or implementation. Public inheritance is a substitutability claim, not merely code reuse.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

57.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch057.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 57: Inheritance
#include <iostream>

int main() {
    std::cout << "Chapter 57: Inheritance" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

57.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

57.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses inheritance and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

57.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

57.6 Summary and further study

Inheritance is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Runtime Polymorphism

Learning objectives

- Explain the central problem addressed by Runtime Polymorphism and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Runtime Polymorphism; contract; representation; lifetime; invariant; diagnostic; trade-off.

58.1 The problem and the mental model

Runtime polymorphism selects an operation through a dynamic type. It is useful when the set of concrete types varies at runtime and a stable interface matters.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

58.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch058.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 58: Runtime Polymorphism
#include <iostream>
#include <memory>
#include <vector>

struct Shape {
    virtual ~Shape() = default;
    virtual int area() const = 0;
};

struct Square final : Shape {
    explicit Square(int side) : side(side) {}
    int area() const override { return side * side; }
    int side;
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Square>(5));
    std::cout << shapes.front()->area() << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

58.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

58.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses runtime polymorphism and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

58.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

58.6 Summary and further study

Runtime Polymorphism is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Abstract Interfaces

Learning objectives

- Explain the central problem addressed by Abstract Interfaces and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Abstract Interfaces; contract; representation; lifetime; invariant; diagnostic; trade-off.

59.1 The problem and the mental model

An abstract interface states capabilities without choosing a representation. Small interfaces reduce accidental coupling and make test doubles more practical.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

59.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch059.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 59: Abstract Interfaces
#include <iostream>

int main() {
    std::cout << "Chapter 59: Abstract Interfaces" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

59.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

59.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses abstract interfaces and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

59.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

59.6 Summary and further study

Abstract Interfaces is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Virtual Functions

Learning objectives

- Explain the central problem addressed by Virtual Functions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Virtual Functions; contract; representation; lifetime; invariant; diagnostic; trade-off.

60.1 The problem and the mental model

Virtual dispatch enables runtime polymorphism but introduces lifetime and destruction requirements. A polymorphic base normally needs a virtual destructor when deleted through the base interface.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

60.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch060.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 60: Virtual Functions
#include <iostream>

int main() {
    std::cout << "Chapter 60: Virtual Functions" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

60.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

60.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses virtual functions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

60.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

60.6 Summary and further study

Virtual Functions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Object Slicing

Learning objectives

- Explain the central problem addressed by Object Slicing and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Object Slicing; contract; representation; lifetime; invariant; diagnostic; trade-off.

61.1 The problem and the mental model

Copying a derived object into a base value discards the derived part. References, pointers, and value-oriented alternatives avoid slicing when polymorphism is intended.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

61.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch061.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 61: Object Slicing
#include <iostream>

int main() {
    std::cout << "Chapter 61: Object Slicing" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

61.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

61.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses object slicing and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

61.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

61.6 Summary and further study

Object Slicing is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Multiple Inheritance

Learning objectives

- Explain the central problem addressed by Multiple Inheritance and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Multiple Inheritance; contract; representation; lifetime; invariant; diagnostic; trade-off.

62.1 The problem and the mental model

Multiple inheritance can model independent interfaces or shared implementation, but ambiguity and layout complexity increase. It is most defensible when each base has a clear role.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

62.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch062.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 62: Multiple Inheritance
#include <iostream>

int main() {
    std::cout << "Chapter 62: Multiple Inheritance" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

62.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

62.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses multiple inheritance and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

62.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

62.6 Summary and further study

Multiple Inheritance is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Alternatives to Inheritance

Learning objectives

- Explain the central problem addressed by Alternatives to Inheritance and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 4.

Key terminology: Alternatives to Inheritance; contract; representation; lifetime; invariant; diagnostic; trade-off.

63.1 The problem and the mental model

Variants, concepts, type erasure, composition, and tagged data can express variation without a class hierarchy. The right choice depends on when the set of alternatives is known and who owns extension.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

63.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch063.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 63: Alternatives to Inheritance
#include <iostream>

int main() {
    std::cout << "Chapter 63: Alternatives to Inheritance" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

63.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

63.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses alternatives to inheritance and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

63.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

63.6 Summary and further study

Alternatives to Inheritance is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART V

The Standard Library

Part 5: The Standard Library

Reusable, specified components for data, algorithms, time, files, and common program infrastructure.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

The Purpose and Structure of the Standard Library

Learning objectives

- Explain the central problem addressed by The Purpose and Structure of the Standard Library and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: The Purpose and Structure of the Standard Library; contract; representation; lifetime; invariant; diagnostic; trade-off.

64.1 The problem and the mental model

The standard library supplies vocabulary types, containers, algorithms, utilities, and I/O. Learning its structure prevents unnecessary reinvention and makes code legible to other C++ programmers.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

64.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch064.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 64: The Purpose and Structure of the Standard Library
#include <iostream>

int main() {
    std::cout << "Chapter 64: The Purpose and Structure of the Standard Library" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

64.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

64.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses the purpose and structure of the standard library and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

64.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

64.6 Summary and further study

The Purpose and Structure of the Standard Library is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Containers

Learning objectives

- Explain the central problem addressed by Containers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Containers; contract; representation; lifetime; invariant; diagnostic; trade-off.

65.1 The problem and the mental model

Containers define ownership, access patterns, complexity, and invalidation rules. Choosing one is a data-access decision, not merely a syntax decision.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

65.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch065.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 65: Containers
#include <iostream>

int main() {
    std::cout << "Chapter 65: Containers" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

65.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

65.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses containers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

65.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

65.6 Summary and further study

Containers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Iterators

Learning objectives

- Explain the central problem addressed by Iterators and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Iterators; contract; representation; lifetime; invariant; diagnostic; trade-off.

66.1 The problem and the mental model

Iterators generalize traversal across containers and ranges. Their validity and category determine which algorithms are legal and what complexity they can achieve.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

66.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch066.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 66: Iterators
#include <iostream>

int main() {
    std::cout << "Chapter 66: Iterators" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

66.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

66.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses iterators and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

66.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

66.6 Summary and further study

Iterators is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Algorithms

Learning objectives

- Explain the central problem addressed by Algorithms and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Algorithms; contract; representation; lifetime; invariant; diagnostic; trade-off.

67.1 The problem and the mental model

Algorithms separate what should be done from how a container stores elements. Preconditions, projections, predicates, and complexity are part of an algorithm's contract.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

67.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch067.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 67: Algorithms
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values{4, 1, 3, 2};
    std::sort(values.begin(), values.end());
    for (const int value : values) { std::cout << value << " "; }
    std::cout << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

67.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

67.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses algorithms and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

67.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

67.6 Summary and further study

Algorithms is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Ranges and Views

Learning objectives

- Explain the central problem addressed by Ranges and Views and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Ranges and Views; contract; representation; lifetime; invariant; diagnostic; trade-off.

68.1 The problem and the mental model

Ranges compose algorithms and views with less iterator plumbing. Views are usually lazy and non-owning, so lifetime and repeated evaluation remain design concerns.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

68.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch068.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 68: Ranges and Views
#include <iostream>

int main() {
    std::cout << "Chapter 68: Ranges and Views" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

68.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

68.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses ranges and views and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

68.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

68.6 Summary and further study

Ranges and Views is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Function Objects

Learning objectives

- Explain the central problem addressed by Function Objects and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Function Objects; contract; representation; lifetime; invariant; diagnostic; trade-off.

69.1 The problem and the mental model

Callable objects include functions, pointers to functions, lambdas, and types with `operator()`. They let algorithms receive behavior without committing to one global name.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

69.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch069.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 69: Function Objects
#include <iostream>

int main() {
    std::cout << "Chapter 69: Function Objects" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

69.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

69.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses function objects and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

69.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

69.6 Summary and further study

Function Objects is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Lambda Expressions

Learning objectives

- Explain the central problem addressed by Lambda Expressions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Lambda Expressions; contract; representation; lifetime; invariant; diagnostic; trade-off.

70.1 The problem and the mental model

Lambdas create local callable objects with explicit capture and parameter rules. Capture is a lifetime decision, especially when a callable escapes its defining scope.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

70.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch070.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 70: Lambda Expressions
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    const std::vector<int> values{1, 2, 3, 4};
    const auto even = std::count_if(values.begin(), values.end(), [](int value) {
        return value % 2 == 0;
    });
    std::cout << even << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

70.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

70.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses lambda expressions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

70.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

70.6 Summary and further study

Lambda Expressions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Associative Containers

Learning objectives

- Explain the central problem addressed by Associative Containers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Associative Containers; contract; representation; lifetime; invariant; diagnostic; trade-off.

71.1 The problem and the mental model

Ordered maps and sets use keys and ordering to provide logarithmic operations under their specified complexity model. Key comparability and iterator stability shape their use.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

71.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch071.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 71: Associative Containers
#include <iostream>

int main() {
    std::cout << "Chapter 71: Associative Containers" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

71.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

71.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses associative containers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

71.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

71.6 Summary and further study

Associative Containers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Unordered Containers

Learning objectives

- Explain the central problem addressed by Unordered Containers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Unordered Containers; contract; representation; lifetime; invariant; diagnostic; trade-off.

72.1 The problem and the mental model

Hash tables use a hash function and an equality relation. Average-case performance depends on distribution and load, while worst-case behavior remains part of robust design.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

72.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch072.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 72: Unordered Containers
#include <iostream>

int main() {
    std::cout << "Chapter 72: Unordered Containers" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

72.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

72.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses unordered containers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

72.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

72.6 Summary and further study

Unordered Containers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Stacks, Queues, and Priority Queues

Learning objectives

- Explain the central problem addressed by Stacks, Queues, and Priority Queues and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Stacks, Queues, and Priority Queues; contract; representation; lifetime; invariant; diagnostic; trade-off.

73.1 The problem and the mental model

Adaptors expose restricted interfaces for disciplined access patterns. Restricting operations can make invariants and intent clearer.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

73.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch073.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 73: Stacks, Queues, and Priority Queues
#include <iostream>

int main() {
    std::cout << "Chapter 73: Stacks, Queues, and Priority Queues" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

73.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

73.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses stacks, queues, and priority queues and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

73.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

73.6 Summary and further study

Stacks, Queues, and Priority Queues is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Optional Values and Variants

Learning objectives

- Explain the central problem addressed by Optional Values and Variants and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Optional Values and Variants; contract; representation; lifetime; invariant; diagnostic; trade-off.

74.1 The problem and the mental model

optional represents a value that may be absent; variant represents one active alternative from a closed set. Both make state explicit without sentinel values.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

74.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch074.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 74: Optional Values and Variants
#include <iostream>
#include <variant>
#include <string>

int main() {
    std::variant<int, std::string> result = std::string{"ready"};
    std::visit([](const auto& value) { std::cout << value << "\n"; }, result);
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

74.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

74.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses optional values and variants and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

74.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

74.6 Summary and further study

Optional Values and Variants is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Tuples

Learning objectives

- Explain the central problem addressed by Tuples and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Tuples; contract; representation; lifetime; invariant; diagnostic; trade-off.

75.1 The problem and the mental model

Tuples group heterogeneous values without naming a domain type. They are useful for local composition but can become opaque at public API boundaries.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

75.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch075.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 75: Tuples
#include <iostream>

int main() {
    std::cout << "Chapter 75: Tuples" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

75.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

75.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses tuples and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

75.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

75.6 Summary and further study

Tuples is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Time and Clocks

Learning objectives

- Explain the central problem addressed by Time and Clocks and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Time and Clocks; contract; representation; lifetime; invariant; diagnostic; trade-off.

76.1 The problem and the mental model

A duration is an amount of time; a time point is a position on a clock. Steady clocks are appropriate for elapsed-time measurement; wall clocks can jump.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

76.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch076.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 76: Time and Clocks
#include <iostream>

int main() {
    std::cout << "Chapter 76: Time and Clocks" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

76.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

76.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses time and clocks and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

76.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

76.6 Summary and further study

Time and Clocks is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Random-Number Facilities

Learning objectives

- Explain the central problem addressed by Random-Number Facilities and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Random-Number Facilities; contract; representation; lifetime; invariant; diagnostic; trade-off.

77.1 The problem and the mental model

Random engines generate deterministic sequences from state; distributions map those sequences to a desired statistical shape. Reproducibility, seeding, and security requirements must be separated.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

77.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch077.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 77: Random-Number Facilities
#include <iostream>

int main() {
    std::cout << "Chapter 77: Random-Number Facilities" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

77.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

77.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses random-number facilities and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

77.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

77.6 Summary and further study

Random-Number Facilities is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

File Input and Output

Learning objectives

- Explain the central problem addressed by File Input and Output and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: File Input and Output; contract; representation; lifetime; invariant; diagnostic; trade-off.

78.1 The problem and the mental model

File streams connect C++ objects to persistent bytes. Correct file handling checks open and operation states, specifies encoding and format, and separates parsing from storage policy.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

78.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch078.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 78: File Input and Output
#include <fstream>
#include <iostream>
#include <string>

int main() {
    const std::string path = "modern_cpp_example.txt";
    { std::ofstream output(path); output << "portable text\n"; }
    std::ifstream input(path);
    std::string line;
    std::getline(input, line);
    std::cout << line << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

78.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first

reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

78.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses file input and output and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

78.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

78.6 Summary and further study

File Input and Output is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Formatting

Learning objectives

- Explain the central problem addressed by Formatting and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Formatting; contract; representation; lifetime; invariant; diagnostic; trade-off.

79.1 The problem and the mental model

Formatting turns values into human-readable or machine-readable text. Locale, escaping, alignment, and available library support all affect portability and correctness.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

79.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch079.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 79: Formatting
#include <iostream>

int main() {
    std::cout << "Chapter 79: Formatting" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

79.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

79.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses formatting and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

79.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

79.6 Summary and further study

Formatting is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Filesystem Operations

Learning objectives

- Explain the central problem addressed by Filesystem Operations and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 5.

Key terminology: Filesystem Operations; contract; representation; lifetime; invariant; diagnostic; trade-off.

80.1 The problem and the mental model

`std::filesystem` supplies path and directory vocabulary across platforms. Paths are not strings with universal separators, and failures can arise from permissions, races, or missing entries.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

80.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch080.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 80: Filesystem Operations
#include <filesystem>
#include <iostream>

int main() {
    const auto path = std::filesystem::current_path();
    std::cout << path.filename().string() << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

80.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

80.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses filesystem operations and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

80.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

80.6 Summary and further study

Filesystem Operations is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART VI

Generic Programming

Part 6: Generic Programming

Templates and compile-time reasoning as tools for reusable interfaces, not puzzles for their own sake.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Function Templates

Learning objectives

- Explain the central problem addressed by Function Templates and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Function Templates; contract; representation; lifetime; invariant; diagnostic; trade-off.

81.1 The problem and the mental model

A function template describes a family of functions parameterized by types or values. Instantiation produces a concrete specialization checked for the actual arguments.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

81.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch081.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 81: Function Templates
#include <iostream>

template <typename T>
T maximum(T left, T right) {
    return left < right ? right : left;
}

int main() {
    std::cout << maximum(3, 8) << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

81.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

81.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses function templates and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

81.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

81.6 Summary and further study

Function Templates is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Class Templates

Learning objectives

- Explain the central problem addressed by Class Templates and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Class Templates; contract; representation; lifetime; invariant; diagnostic; trade-off.

82.1 The problem and the mental model

A class template describes a family of related types. Its parameters can encode element types, policies, allocators, or compile-time dimensions.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

82.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch082.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 82: Class Templates
#include <iostream>

int main() {
    std::cout << "Chapter 82: Class Templates" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

82.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

82.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses class templates and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

82.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

82.6 Summary and further study

Class Templates is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Template Argument Deduction

Learning objectives

- Explain the central problem addressed by Template Argument Deduction and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Template Argument Deduction; contract; representation; lifetime; invariant; diagnostic; trade-off.

83.1 The problem and the mental model

Deduction matches argument types against a template pattern. References, cv-qualification, arrays, and conversions explain many surprising results.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

83.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch083.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 83: Template Argument Deduction
#include <iostream>

int main() {
    std::cout << "Chapter 83: Template Argument Deduction" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

83.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

83.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses template argument deduction and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

83.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

83.6 Summary and further study

Template Argument Deduction is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Concepts and Constraints

Learning objectives

- Explain the central problem addressed by Concepts and Constraints and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Concepts and Constraints; contract; representation; lifetime; invariant; diagnostic; trade-off.

84.1 The problem and the mental model

Concepts state requirements on template arguments in a readable form. Constraints improve overload selection and diagnostics, but they do not replace semantic documentation.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

84.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch084.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 84: Concepts and Constraints
#include <concepts>
#include <iostream>

template <std::integral T>
T add(T left, T right) { return left + right; }

int main() {
    std::cout << add(2, 3) << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

84.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal,

ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

84.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses concepts and constraints and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

84.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

84.6 Summary and further study

Concepts and Constraints is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Variadic Templates

Learning objectives

- Explain the central problem addressed by Variadic Templates and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Variadic Templates; contract; representation; lifetime; invariant; diagnostic; trade-off.

85.1 The problem and the mental model

Parameter packs represent an arbitrary number of arguments or types. Fold expressions make common reductions concise, while recursive designs remain useful for some structural problems.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

85.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch085.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 85: Variadic Templates
#include <iostream>

int main() {
    std::cout << "Chapter 85: Variadic Templates" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

85.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

85.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses variadic templates and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

85.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

85.6 Summary and further study

Variadic Templates is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Compile-Time Programming

Learning objectives

- Explain the central problem addressed by Compile-Time Programming and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Compile-Time Programming; contract; representation; lifetime; invariant; diagnostic; trade-off.

86.1 The problem and the mental model

Compile-time programming moves selected work into translation. The benefit is an earlier check or a generated specialization, not a blanket claim that compile-time code is always faster.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

86.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch086.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 86: Compile-Time Programming
#include <iostream>

int main() {
    std::cout << "Chapter 86: Compile-Time Programming" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

86.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

86.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses compile-time programming and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

86.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

86.6 Summary and further study

Compile-Time Programming is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

constexpr Evaluation

Learning objectives

- Explain the central problem addressed by constexpr Evaluation and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: constexpr Evaluation; contract; representation; lifetime; invariant; diagnostic; trade-off.

87.1 The problem and the mental model

constexpr permits evaluation during constant evaluation when all rules are satisfied. The same function can often remain usable at runtime, which supports one source of truth.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

87.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch087.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 87: constexpr Evaluation
#include <iostream>

int main() {
    std::cout << "Chapter 87: constexpr Evaluation" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

87.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

87.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses `constexpr` evaluation and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

87.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

87.6 Summary and further study

`constexpr` Evaluation is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Type Traits

Learning objectives

- Explain the central problem addressed by Type Traits and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Type Traits; contract; representation; lifetime; invariant; diagnostic; trade-off.

88.1 The problem and the mental model

Type traits query or transform type properties. They are building blocks for generic constraints and adaptation, but a named concept is often clearer at an API boundary.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

88.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch088.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 88: Type Traits
#include <iostream>

int main() {
    std::cout << "Chapter 88: Type Traits" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

88.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

88.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses type traits and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

88.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

88.6 Summary and further study

Type Traits is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Generic Algorithms

Learning objectives

- Explain the central problem addressed by Generic Algorithms and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Generic Algorithms; contract; representation; lifetime; invariant; diagnostic; trade-off.

89.1 The problem and the mental model

A generic algorithm states its needed operations and works across many types. Correctness depends on documenting semantic requirements that syntax alone cannot express.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

89.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch089.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 89: Generic Algorithms
#include <iostream>

int main() {
    std::cout << "Chapter 89: Generic Algorithms" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

89.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

89.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses generic algorithms and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

89.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

89.6 Summary and further study

Generic Algorithms is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Template Errors and Diagnostics

Learning objectives

- Explain the central problem addressed by Template Errors and Diagnostics and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 6.

Key terminology: Template Errors and Diagnostics; contract; representation; lifetime; invariant; diagnostic; trade-off.

90.1 The problem and the mental model

Template diagnostics can be long because the compiler reports substitutions and overload candidates. Reducing the example and adding constraints makes failures actionable.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

90.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch090.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 90: Template Errors and Diagnostics
#include <iostream>

int main() {
    std::cout << "Chapter 90: Template Errors and Diagnostics" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

90.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

90.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses template errors and diagnostics and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

90.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

90.6 Summary and further study

Template Errors and Diagnostics is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART VII

Errors, Testing, and Reliability

Part 7: Errors, Testing, and Reliability

Failure modes, evidence, and tools for finding defects before users do.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Syntax, Linker, Logic, and Runtime Errors

Learning objectives

- Explain the central problem addressed by Syntax, Linker, Logic, and Runtime Errors and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Syntax, Linker, Logic, and Runtime Errors; contract; representation; lifetime; invariant; diagnostic; trade-off.

91.1 The problem and the mental model

Different failure classes arise at different stages. Classification narrows the search and prevents treating every failure as a compiler problem.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

91.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch091.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 91: Syntax, Linker, Logic, and Runtime Errors
#include <iostream>

int main() {
    std::cout << "Chapter 91: Syntax, Linker, Logic, and Runtime Errors" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

91.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

91.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses syntax, linker, logic, and runtime errors and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

91.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

91.6 Summary and further study

Syntax, Linker, Logic, and Runtime Errors is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Assertions

Learning objectives

- Explain the central problem addressed by Assertions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Assertions; contract; representation; lifetime; invariant; diagnostic; trade-off.

92.1 The problem and the mental model

Assertions state conditions expected to be true when a defect would indicate a programming error. They complement, rather than replace, handling of ordinary external failures.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

92.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch092.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 92: Assertions
#include <iostream>

int main() {
    std::cout << "Chapter 92: Assertions" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

92.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

92.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses assertions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

92.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

92.6 Summary and further study

Assertions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Exceptions

Learning objectives

- Explain the central problem addressed by Exceptions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Exceptions; contract; representation; lifetime; invariant; diagnostic; trade-off.

93.1 The problem and the mental model

Exceptions transfer control out of a failing operation. Good exception design names the boundary, preserves invariants, and avoids using exceptions as invisible ordinary control flow.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

93.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch093.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 93: Exceptions
#include <iostream>
#include <stdexcept>

int divide(int numerator, int denominator) {
    if (denominator == 0) { throw std::invalid_argument{"zero denominator"}; }
    return numerator / denominator;
}

int main() {
    try { std::cout << divide(8, 2) << "\n"; }
    catch (const std::exception& error) { std::cerr << error.what() << "\n"; return 1; }
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

93.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

93.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses exceptions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

93.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

93.6 Summary and further study

Exceptions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Error Codes

Learning objectives

- Explain the central problem addressed by Error Codes and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Error Codes; contract; representation; lifetime; invariant; diagnostic; trade-off.

94.1 The problem and the mental model

Error codes make failure part of a value-level protocol. They are explicit and useful at low-level or performance-sensitive boundaries, but callers must not ignore them.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

94.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch094.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 94: Error Codes
#include <iostream>

int main() {
    std::cout << "Chapter 94: Error Codes" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

94.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

94.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses error codes and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

94.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

94.6 Summary and further study

Error Codes is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Expected-Style Error Handling

Learning objectives

- Explain the central problem addressed by Expected-Style Error Handling and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Expected-Style Error Handling; contract; representation; lifetime; invariant; diagnostic; trade-off.

95.1 The problem and the mental model

An expected-style result carries either a value or an error. It makes recoverable failure visible in the type and composes well when the caller can act locally.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

95.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch095.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 95: Expected-Style Error Handling
#include <iostream>
#include <string>
#include <type_traits>
#include <variant>

struct Error { std::string message; };
using Result = std::variant<int, Error>;

Result parse_positive(int value) {
    if (value < 0) return Error{"negative value"};
    return value;
}

int main() {
    const Result result = parse_positive(4);
    std::visit([&](const auto& value) {
        if constexpr (std::is_same_v<std::decay_t<decltype(value)>, Error>) std::cout << value.message << "\n";
        else std::cout << value << "\n";
    }, result);
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

95.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

95.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses expected-style error handling and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

95.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

95.6 Summary and further study

Expected-Style Error Handling is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Exception Safety

Learning objectives

- Explain the central problem addressed by Exception Safety and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Exception Safety; contract; representation; lifetime; invariant; diagnostic; trade-off.

96.1 The problem and the mental model

The no-throw, strong, and basic guarantees describe what remains true when an operation fails. These guarantees are design properties of operations, not decorative labels.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

96.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch096.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 96: Exception Safety
#include <iostream>

int main() {
    std::cout << "Chapter 96: Exception Safety" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

96.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

96.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses exception safety and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

96.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

96.6 Summary and further study

Exception Safety is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Unit Testing

Learning objectives

- Explain the central problem addressed by Unit Testing and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Unit Testing; contract; representation; lifetime; invariant; diagnostic; trade-off.

97.1 The problem and the mental model

Unit tests isolate a small behavior and make its contract executable. A good test fails for a meaningful reason and remains independent of unrelated timing or global state.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

97.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch097.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 97: Unit Testing
#include <iostream>

int main() {
    std::cout << "Chapter 97: Unit Testing" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

97.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

97.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses unit testing and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

97.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

97.6 Summary and further study

Unit Testing is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Integration Testing

Learning objectives

- Explain the central problem addressed by Integration Testing and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Integration Testing; contract; representation; lifetime; invariant; diagnostic; trade-off.

98.1 The problem and the mental model

Integration tests exercise boundaries among real components. They catch configuration, serialization, file, and lifecycle defects that unit tests intentionally exclude.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

98.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch098.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 98: Integration Testing
#include <iostream>

int main() {
    std::cout << "Chapter 98: Integration Testing" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

98.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

98.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses integration testing and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

98.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

98.6 Summary and further study

Integration Testing is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Property-Based Testing

Learning objectives

- Explain the central problem addressed by Property-Based Testing and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Property-Based Testing; contract; representation; lifetime; invariant; diagnostic; trade-off.

99.1 The problem and the mental model

Property-based testing generates many inputs to check general relationships. The property must be a real invariant, not a restatement of the implementation.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

99.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch099.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 99: Property-Based Testing
#include <iostream>

int main() {
    std::cout << "Chapter 99: Property-Based Testing" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

99.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

99.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses property-based testing and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

99.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

99.6 Summary and further study

Property-Based Testing is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Test Doubles

Learning objectives

- Explain the central problem addressed by Test Doubles and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Test Doubles; contract; representation; lifetime; invariant; diagnostic; trade-off.

100.1 The problem and the mental model

Stubs, fakes, spies, and mocks replace collaborators for a test. They are useful when a boundary is expensive or nondeterministic, but excessive mocking can test a design's seams rather than its behavior.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

100.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch100.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 100: Test Doubles
#include <iostream>

int main() {
    std::cout << "Chapter 100: Test Doubles" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

100.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

100.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses test doubles and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

100.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

100.6 Summary and further study

Test Doubles is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Debuggers

Learning objectives

- Explain the central problem addressed by Debuggers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Debuggers; contract; representation; lifetime; invariant; diagnostic; trade-off.

101.1 The problem and the mental model

A debugger observes a running process through breakpoints, stack frames, variables, and watchpoints. Observation changes timing, so a debugger is evidence, not an infallible explanation of concurrent behavior.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

101.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch101.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 101: Debuggers
#include <iostream>

int main() {
    std::cout << "Chapter 101: Debuggers" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

101.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

101.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses debuggers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

101.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

101.6 Summary and further study

Debuggers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Sanitizers

Learning objectives

- Explain the central problem addressed by Sanitizers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Sanitizers; contract; representation; lifetime; invariant; diagnostic; trade-off.

102.1 The problem and the mental model

Address, undefined-behavior, and thread sanitizers instrument programs to catch classes of defects. They increase cost and coverage needs but often turn mysterious symptoms into local failures.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

102.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch102.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 102: Sanitizers
#include <iostream>

int main() {
    std::cout << "Chapter 102: Sanitizers" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

102.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

102.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses sanitizers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

102.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

102.6 Summary and further study

Sanitizers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Static Analysis

Learning objectives

- Explain the central problem addressed by Static Analysis and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Static Analysis; contract; representation; lifetime; invariant; diagnostic; trade-off.

103.1 The problem and the mental model

Static analyzers inspect code without executing every path. Warnings are hypotheses that require triage, not proof that every reported line is wrong.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

103.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch103.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 103: Static Analysis
#include <iostream>

int main() {
    std::cout << "Chapter 103: Static Analysis" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

103.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

103.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses static analysis and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

103.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

103.6 Summary and further study

Static Analysis is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Logging

Learning objectives

- Explain the central problem addressed by Logging and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Logging; contract; representation; lifetime; invariant; diagnostic; trade-off.

104.1 The problem and the mental model

Logs record selected events and context for diagnosis. Levels, stable fields, privacy, volume, and correlation identifiers determine whether logs are useful in production.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

104.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch104.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 104: Logging
#include <iostream>

int main() {
    std::cout << "Chapter 104: Logging" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

104.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

104.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses logging and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

104.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

104.6 Summary and further study

Logging is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Reproducing and Minimizing Bugs

Learning objectives

- Explain the central problem addressed by Reproducing and Minimizing Bugs and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Reproducing and Minimizing Bugs; contract; representation; lifetime; invariant; diagnostic; trade-off.

105.1 The problem and the mental model

A reproducible failure is a scientific foothold. Reduce inputs, environment, and timing until the smallest case still demonstrates the defect.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

105.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch105.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 105: Reproducing and Minimizing Bugs
#include <iostream>

int main() {
    std::cout << "Chapter 105: Reproducing and Minimizing Bugs" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

105.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

105.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses reproducing and minimizing bugs and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

105.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

105.6 Summary and further study

Reproducing and Minimizing Bugs is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Defensive Programming

Learning objectives

- Explain the central problem addressed by Defensive Programming and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 7.

Key terminology: Defensive Programming; contract; representation; lifetime; invariant; diagnostic; trade-off.

106.1 The problem and the mental model

Defensive programming makes assumptions explicit at boundaries and preserves invariants. It should improve failure behavior without obscuring the actual contract.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

106.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch106.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 106: Defensive Programming
#include <iostream>

int main() {
    std::cout << "Chapter 106: Defensive Programming" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

106.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

106.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses defensive programming and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

106.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

106.6 Summary and further study

Defensive Programming is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART VIII

Data Structures and Algorithms

Part 8: Data Structures and Algorithms

Ways to represent problems, compare costs, and select algorithms using evidence.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Algorithmic Thinking

Learning objectives

- Explain the central problem addressed by Algorithmic Thinking and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Algorithmic Thinking; contract; representation; lifetime; invariant; diagnostic; trade-off.

107.1 The problem and the mental model

Algorithmic thinking names state, operations, invariants, termination, and cost before code. It turns a vague task into a sequence that can be reviewed and tested.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

107.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch107.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 107: Algorithmic Thinking
#include <iostream>

int main() {
    std::cout << "Chapter 107: Algorithmic Thinking" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

107.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

107.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses algorithmic thinking and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

107.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

107.6 Summary and further study

Algorithmic Thinking is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Measuring Time and Space

Learning objectives

- Explain the central problem addressed by Measuring Time and Space and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Measuring Time and Space; contract; representation; lifetime; invariant; diagnostic; trade-off.

108.1 The problem and the mental model

Time and space measurements depend on a workload, environment, and measurement method. A number without those conditions is not a portable fact.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

108.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch108.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 108: Measuring Time and Space
#include <iostream>

int main() {
    std::cout << "Chapter 108: Measuring Time and Space" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

108.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

108.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses measuring time and space and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

108.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

108.6 Summary and further study

Measuring Time and Space is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Asymptotic Complexity

Learning objectives

- Explain the central problem addressed by Asymptotic Complexity and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Asymptotic Complexity; contract; representation; lifetime; invariant; diagnostic; trade-off.

109.1 The problem and the mental model

Asymptotic notation describes how resource use grows as input size grows. It abstracts constants and hardware, which is useful for scaling questions but insufficient for every engineering choice.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

109.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch109.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 109: Asymptotic Complexity
#include <iostream>

int main() {
    std::cout << "Chapter 109: Asymptotic Complexity" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

109.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

109.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses asymptotic complexity and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

109.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

109.6 Summary and further study

Asymptotic Complexity is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Recursion

Learning objectives

- Explain the central problem addressed by Recursion and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Recursion; contract; representation; lifetime; invariant; diagnostic; trade-off.

110.1 The problem and the mental model

Recursion solves a problem through smaller instances and a base case. Correctness requires progress toward the base case; performance requires understanding call depth and repeated work.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

110.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch110.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 110: Recursion
#include <iostream>

int main() {
    std::cout << "Chapter 110: Recursion" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

110.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

110.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses recursion and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

110.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

110.6 Summary and further study

Recursion is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Searching

Learning objectives

- Explain the central problem addressed by Searching and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Searching; contract; representation; lifetime; invariant; diagnostic; trade-off.

111.1 The problem and the mental model

Searching strategies exploit different structure: linear scans need little organization, while binary search needs ordering and a precise range invariant.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

111.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch111.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 111: Searching
#include <iostream>

int main() {
    std::cout << "Chapter 111: Searching" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

111.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

111.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses searching and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

111.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

111.6 Summary and further study

Searching is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Sorting

Learning objectives

- Explain the central problem addressed by Sorting and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Sorting; contract; representation; lifetime; invariant; diagnostic; trade-off.

112.1 The problem and the mental model

Sorting establishes an ordering that can simplify later work. Stability, memory use, comparator correctness, and complexity all matter beyond the name of an algorithm.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

112.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch112.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 112: Sorting
#include <algorithm>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> values{9, 4, 7, 1};
    std::sort(values.begin(), values.end());
    std::cout << values.front() << " " << values.back() << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

112.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

112.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses sorting and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

112.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

112.6 Summary and further study

Sorting is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Linked Structures

Learning objectives

- Explain the central problem addressed by Linked Structures and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Linked Structures; contract; representation; lifetime; invariant; diagnostic; trade-off.

113.1 The problem and the mental model

Linked structures connect nodes through references or pointers. They teach ownership and invalidation, but their pointer flexibility can trade away locality and simplicity.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

113.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch113.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 113: Linked Structures
#include <iostream>

int main() {
    std::cout << "Chapter 113: Linked Structures" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

113.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

113.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses linked structures and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

113.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

113.6 Summary and further study

Linked Structures is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Stacks and Queues

Learning objectives

- Explain the central problem addressed by Stacks and Queues and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Stacks and Queues; contract; representation; lifetime; invariant; diagnostic; trade-off.

114.1 The problem and the mental model

Stacks and queues restrict access to express last-in-first-out or first-in-first-out policy. The abstraction often matters more than the underlying container.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

114.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch114.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 114: Stacks and Queues
#include <iostream>

int main() {
    std::cout << "Chapter 114: Stacks and Queues" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

114.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

114.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses stacks and queues and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

114.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

114.6 Summary and further study

Stacks and Queues is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Trees

Learning objectives

- Explain the central problem addressed by Trees and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Trees; contract; representation; lifetime; invariant; diagnostic; trade-off.

115.1 The problem and the mental model

Trees represent hierarchical relationships with acyclic parent-child structure. Balance, ordering, ownership, and traversal determine whether a tree solves the actual problem.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

115.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch115.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 115: Trees
#include <iostream>

int main() {
    std::cout << "Chapter 115: Trees" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

115.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

115.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses trees and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

115.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

115.6 Summary and further study

Trees is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Heaps

Learning objectives

- Explain the central problem addressed by Heaps and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Heaps; contract; representation; lifetime; invariant; diagnostic; trade-off.

116.1 The problem and the mental model

A heap maintains a partial order that makes an extreme element accessible. It is useful for priority queues and selection, not for producing a fully sorted sequence by itself.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

116.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch116.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 116: Heaps
#include <iostream>

int main() {
    std::cout << "Chapter 116: Heaps" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

116.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

116.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses heaps and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

116.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

116.6 Summary and further study

Heaps is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Hash Tables

Learning objectives

- Explain the central problem addressed by Hash Tables and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Hash Tables; contract; representation; lifetime; invariant; diagnostic; trade-off.

117.1 The problem and the mental model

Hash tables map keys to buckets using a hash function. Correct equality and hash consistency are mandatory; collisions and load determine observed performance.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

117.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch117.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 117: Hash Tables
#include <iostream>

int main() {
    std::cout << "Chapter 117: Hash Tables" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

117.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

117.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses hash tables and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

117.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

117.6 Summary and further study

Hash Tables is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Graphs

Learning objectives

- Explain the central problem addressed by Graphs and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Graphs; contract; representation; lifetime; invariant; diagnostic; trade-off.

118.1 The problem and the mental model

Graphs model entities and relationships. Choosing directedness, weights, and representation precedes selecting traversal or shortest-path algorithms.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

118.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch118.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 118: Graphs
#include <iostream>
#include <cstdint>
#include <queue>
#include <vector>

int main() {
    const std::vector<std::vector<std::size_t>> graph{{1, 2}, {0, 3}, {0}, {1}};
    std::vector<bool> seen(graph.size());
    std::queue<std::size_t> pending;
    pending.push(0); seen[0] = true;
    while (!pending.empty()) {
        const std::size_t node = pending.front(); pending.pop();
        std::cout << node << " ";
        for (const std::size_t next : graph[node]) if (!seen[next]) { seen[next] = true; pending.push(next); }
    }
    std::cout << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

118.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

118.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses graphs and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

118.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

118.6 Summary and further study

Graphs is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Dynamic Programming

Learning objectives

- Explain the central problem addressed by Dynamic Programming and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Dynamic Programming; contract; representation; lifetime; invariant; diagnostic; trade-off.

119.1 The problem and the mental model

Dynamic programming stores solutions to overlapping subproblems. A recurrence, state definition, base case, and evaluation order make the optimization auditable.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

119.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch119.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 119: Dynamic Programming
#include <iostream>

int main() {
    std::cout << "Chapter 119: Dynamic Programming" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

119.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

119.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses dynamic programming and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

119.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

119.6 Summary and further study

Dynamic Programming is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Greedy Algorithms

Learning objectives

- Explain the central problem addressed by Greedy Algorithms and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Greedy Algorithms; contract; representation; lifetime; invariant; diagnostic; trade-off.

120.1 The problem and the mental model

Greedy algorithms commit to a locally attractive choice. They are correct only when a proof or established property connects local choices to a global optimum.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

120.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch120.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 120: Greedy Algorithms
#include <iostream>

int main() {
    std::cout << "Chapter 120: Greedy Algorithms" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

120.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

120.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses greedy algorithms and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

120.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

120.6 Summary and further study

Greedy Algorithms is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Algorithm Selection

Learning objectives

- Explain the central problem addressed by Algorithm Selection and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Algorithm Selection; contract; representation; lifetime; invariant; diagnostic; trade-off.

121.1 The problem and the mental model

Selection balances contract, constraints, maintainability, data shape, and measured performance. A theoretically optimal algorithm can be the wrong engineering choice for a small or changing workload.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

121.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch121.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 121: Algorithm Selection
#include <iostream>

int main() {
    std::cout << "Chapter 121: Algorithm Selection" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

121.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

121.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses algorithm selection and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

121.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

121.6 Summary and further study

Algorithm Selection is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Benchmarking

Learning objectives

- Explain the central problem addressed by Benchmarking and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 8.

Key terminology: Benchmarking; contract; representation; lifetime; invariant; diagnostic; trade-off.

122.1 The problem and the mental model

A benchmark is an experiment with a workload, harness, environment, and statistical summary. It measures that experiment, not every future use of the code.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

122.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch122.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 122: Benchmarking
#include <iostream>

int main() {
    std::cout << "Chapter 122: Benchmarking" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

122.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

122.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses benchmarking and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

122.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

122.6 Summary and further study

Benchmarking is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART IX

Professional C++ Development

Part 9: Professional C++ Development

Source organization, builds, interfaces, version control, portability, and long-lived software.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Organizing Source and Header Files

Learning objectives

- Explain the central problem addressed by Organizing Source and Header Files and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Organizing Source and Header Files; contract; representation; lifetime; invariant; diagnostic; trade-off.

123.1 The problem and the mental model

Headers expose declarations and reusable definitions; source files hold implementation details. Organization should express ownership and minimize unnecessary recompilation.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

123.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch123.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 123: Organizing Source and Header Files
#include <iostream>

int main() {
    std::cout << "Chapter 123: Organizing Source and Header Files" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

123.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

123.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses organizing source and header files and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

123.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

123.6 Summary and further study

Organizing Source and Header Files is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

One Definition Rule

Learning objectives

- Explain the central problem addressed by One Definition Rule and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: One Definition Rule; contract; representation; lifetime; invariant; diagnostic; trade-off.

124.1 The problem and the mental model

The One Definition Rule constrains which entities may have one or multiple definitions across a program. Violations can compile and still produce ill-formed or fragile binaries.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

124.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch124.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 124: One Definition Rule
#include <iostream>

int main() {
    std::cout << "Chapter 124: One Definition Rule" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

124.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

124.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses one definition rule and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

124.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

124.6 Summary and further study

One Definition Rule is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Separate Compilation

Learning objectives

- Explain the central problem addressed by Separate Compilation and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Separate Compilation; contract; representation; lifetime; invariant; diagnostic; trade-off.

125.1 The problem and the mental model

Separate compilation turns translation units into independently compiled objects. Stable interfaces and dependency boundaries make large builds faster and easier to reason about.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

125.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch125.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 125: Separate Compilation
#include <iostream>

int main() {
    std::cout << "Chapter 125: Separate Compilation" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

125.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

125.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses separate compilation and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

125.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

125.6 Summary and further study

Separate Compilation is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Libraries

Learning objectives

- Explain the central problem addressed by Libraries and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Libraries; contract; representation; lifetime; invariant; diagnostic; trade-off.

126.1 The problem and the mental model

A library packages reusable code as source, static binaries, or shared binaries. Consumers need a compatible interface, build configuration, and ABI story.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

126.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch126.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 126: Libraries
#include <iostream>

int main() {
    std::cout << "Chapter 126: Libraries" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

126.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

126.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses libraries and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

126.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

126.6 Summary and further study

Libraries is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Modules and Their Support Status

Learning objectives

- Explain the central problem addressed by Modules and Their Support Status and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Modules and Their Support Status; contract; representation; lifetime; invariant; diagnostic; trade-off.

127.1 The problem and the mental model

Modules aim to replace textual inclusion with named interfaces and faster, more isolated imports. They are standardized in stages, but toolchain support and build integration remain important practical constraints.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

127.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch127.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 127: Modules and Their Support Status
#include <iostream>

int main() {
    std::cout << "Chapter 127: Modules and Their Support Status" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

127.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

127.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses modules and their support status and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

127.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

127.6 Summary and further study

Modules and Their Support Status is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

CMake

Learning objectives

- Explain the central problem addressed by CMake and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: CMake; contract; representation; lifetime; invariant; diagnostic; trade-off.

128.1 The problem and the mental model

CMake describes targets, properties, dependencies, tests, and installation rather than hard-coding one platform's commands. Generators then produce a native build system.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

128.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch128.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 128: CMake
#include <iostream>

int main() {
    std::cout << "Chapter 128: CMake" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

128.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

128.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses `cmake` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

128.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

128.6 Summary and further study

CMake is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Dependency Management

Learning objectives

- Explain the central problem addressed by Dependency Management and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Dependency Management; contract; representation; lifetime; invariant; diagnostic; trade-off.

129.1 The problem and the mental model

Dependencies add capabilities and also add licensing, update, security, ABI, and reproducibility obligations. A dependency policy is part of system design.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

129.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch129.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 129: Dependency Management
#include <iostream>

int main() {
    std::cout << "Chapter 129: Dependency Management" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

129.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

129.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses dependency management and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

129.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

129.6 Summary and further study

Dependency Management is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Package Managers

Learning objectives

- Explain the central problem addressed by Package Managers and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Package Managers; contract; representation; lifetime; invariant; diagnostic; trade-off.

130.1 The problem and the mental model

Package managers resolve and build external libraries. Locking versions and documenting triplets, toolchains, and feature choices keeps builds reproducible.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

130.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch130.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 130: Package Managers
#include <iostream>

int main() {
    std::cout << "Chapter 130: Package Managers" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

130.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

130.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses package managers and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

130.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

130.6 Summary and further study

Package Managers is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Version Control with Git

Learning objectives

- Explain the central problem addressed by Version Control with Git and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Version Control with Git; contract; representation; lifetime; invariant; diagnostic; trade-off.

131.1 The problem and the mental model

Version control records changes as reviewable history. Small commits, meaningful messages, and reproducible branches reduce the cost of collaboration and recovery.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

131.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch131.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 131: Version Control with Git
#include <iostream>

int main() {
    std::cout << "Chapter 131: Version Control with Git" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

131.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

131.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses version control with git and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

131.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

131.6 Summary and further study

Version Control with Git is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Documentation

Learning objectives

- Explain the central problem addressed by Documentation and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Documentation; contract; representation; lifetime; invariant; diagnostic; trade-off.

132.1 The problem and the mental model

Documentation explains intent, contracts, constraints, and operations. It should answer questions that code alone cannot, without duplicating every implementation detail.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

132.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch132.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 132: Documentation
#include <iostream>

int main() {
    std::cout << "Chapter 132: Documentation" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

132.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

132.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses documentation and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

132.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

132.6 Summary and further study

Documentation is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

API Design

Learning objectives

- Explain the central problem addressed by API Design and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: API Design; contract; representation; lifetime; invariant; diagnostic; trade-off.

133.1 The problem and the mental model

An API is a promise about names, types, ownership, failure, timing, and compatibility. Good APIs make common correct use easy and unsupported use visible.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

133.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch133.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 133: API Design
#include <iostream>

int main() {
    std::cout << "Chapter 133: API Design" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

133.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

133.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses api design and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

133.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

133.6 Summary and further study

API Design is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Coding Conventions

Learning objectives

- Explain the central problem addressed by Coding Conventions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Coding Conventions; contract; representation; lifetime; invariant; diagnostic; trade-off.

134.1 The problem and the mental model

Conventions reduce accidental variation and make review focus on behavior. They are most valuable when automated and kept subordinate to correctness.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

134.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch134.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 134: Coding Conventions
#include <iostream>

int main() {
    std::cout << "Chapter 134: Coding Conventions" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

134.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

134.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses coding conventions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

134.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

134.6 Summary and further study

Coding Conventions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Refactoring

Learning objectives

- Explain the central problem addressed by Refactoring and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Refactoring; contract; representation; lifetime; invariant; diagnostic; trade-off.

135.1 The problem and the mental model

Refactoring changes structure while preserving externally observable behavior. Tests and small steps provide evidence that the intended behavior survived.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

135.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch135.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 135: Refactoring
#include <iostream>

int main() {
    std::cout << "Chapter 135: Refactoring" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

135.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

135.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses refactoring and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

135.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

135.6 Summary and further study

Refactoring is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Code Review

Learning objectives

- Explain the central problem addressed by Code Review and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Code Review; contract; representation; lifetime; invariant; diagnostic; trade-off.

136.1 The problem and the mental model

Code review is a second line of reasoning about correctness, risk, and maintainability. Review comments should identify evidence and consequences rather than preferences alone.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

136.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch136.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 136: Code Review
#include <iostream>

int main() {
    std::cout << "Chapter 136: Code Review" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

136.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

136.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses code review and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

136.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

136.6 Summary and further study

Code Review is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Continuous Integration

Learning objectives

- Explain the central problem addressed by Continuous Integration and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Continuous Integration; contract; representation; lifetime; invariant; diagnostic; trade-off.

137.1 The problem and the mental model

Continuous integration builds and tests changes in a clean, repeatable environment. It is a feedback system, not a substitute for design or local verification.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

137.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch137.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 137: Continuous Integration
#include <iostream>

int main() {
    std::cout << "Chapter 137: Continuous Integration" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

137.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

137.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses continuous integration and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

137.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

137.6 Summary and further study

Continuous Integration is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Portability

Learning objectives

- Explain the central problem addressed by Portability and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Portability; contract; representation; lifetime; invariant; diagnostic; trade-off.

138.1 The problem and the mental model

Portable code depends on specified behavior and isolates platform assumptions. Portability is a property of an interface and build process, not just of source syntax.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

138.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch138.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 138: Portability
#include <iostream>

int main() {
    std::cout << "Chapter 138: Portability" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

138.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

138.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses portability and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

138.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

138.6 Summary and further study

Portability is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Cross-Platform Development

Learning objectives

- Explain the central problem addressed by Cross-Platform Development and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Cross-Platform Development; contract; representation; lifetime; invariant; diagnostic; trade-off.

139.1 The problem and the mental model

Operating systems differ in paths, processes, sockets, time, signals, and toolchains. A portable core plus narrow adapters often contains the differences cleanly.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

139.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch139.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 139: Cross-Platform Development
#include <iostream>

int main() {
    std::cout << "Chapter 139: Cross-Platform Development" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

139.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

139.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses cross-platform development and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

139.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

139.6 Summary and further study

Cross-Platform Development is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Application Binary Interfaces

Learning objectives

- Explain the central problem addressed by Application Binary Interfaces and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Application Binary Interfaces; contract; representation; lifetime; invariant; diagnostic; trade-off.

140.1 The problem and the mental model

An ABI covers calling conventions, object layout, name mangling, exception boundaries, and binary compatibility. Source compatibility does not imply ABI compatibility.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

140.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch140.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 140: Application Binary Interfaces
#include <iostream>

int main() {
    std::cout << "Chapter 140: Application Binary Interfaces" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

140.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

140.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses application binary interfaces and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

140.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

140.6 Summary and further study

Application Binary Interfaces is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Interoperability with C

Learning objectives

- Explain the central problem addressed by Interoperability with C and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Interoperability with C; contract; representation; lifetime; invariant; diagnostic; trade-off.

141.1 The problem and the mental model

C interoperability requires a compatible data and calling boundary, usually with extern "C" and C-compatible types. Ownership and error conventions must cross the boundary explicitly.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

141.2 A small working program

The complete standalone program for this chapter is stored at examples/chapters/ch141.cpp and is compiled by the verification script.

```
// Modern C++ - Chapter 141: Interoperability with C
#include <iostream>

int main() {
    std::cout << "Chapter 141: Interoperability with C" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

141.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

141.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses interoperability with `c` and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

141.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

141.6 Summary and further study

Interoperability with C is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Interoperability with Other Languages

Learning objectives

- Explain the central problem addressed by Interoperability with Other Languages and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 9.

Key terminology: Interoperability with Other Languages; contract; representation; lifetime; invariant; diagnostic; trade-off.

142.1 The problem and the mental model

Language interoperability is an interface-design problem involving data representation, calling conventions, ownership, errors, and packaging. A narrow C ABI is often a useful stable boundary.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

142.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch142.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 142: Interoperability with Other Languages
#include <iostream>

int main() {
    std::cout << "Chapter 142: Interoperability with Other Languages" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

142.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

142.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses interoperability with other languages and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

142.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

142.6 Summary and further study

Interoperability with Other Languages is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART X

Concurrency

Part 10: Concurrency

Processes, threads, synchronization, atomicity, and the memory model as correctness topics.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Processes and Threads

Learning objectives

- Explain the central problem addressed by Processes and Threads and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Processes and Threads; contract; representation; lifetime; invariant; diagnostic; trade-off.

143.1 The problem and the mental model

Processes isolate address spaces and resources; threads share an address space within a process. The distinction changes failure, communication, and synchronization costs.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

143.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch143.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 143: Processes and Threads
#include <iostream>
#include <thread>

int main() {
    std::jthread worker([] { std::cout << "worker\n"; });
    std::cout << "main\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

143.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

143.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses processes and threads and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

143.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

143.6 Summary and further study

Processes and Threads is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Race Conditions

Learning objectives

- Explain the central problem addressed by Race Conditions and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Race Conditions; contract; representation; lifetime; invariant; diagnostic; trade-off.

144.1 The problem and the mental model

A race exists when correctness depends on an uncontrolled ordering of operations. A program can appear correct for thousands of runs and still contain a race.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

144.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch144.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 144: Race Conditions
#include <iostream>

int main() {
    std::cout << "Chapter 144: Race Conditions" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

144.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

144.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses race conditions and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

144.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

144.6 Summary and further study

Race Conditions is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Mutual Exclusion

Learning objectives

- Explain the central problem addressed by Mutual Exclusion and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Mutual Exclusion; contract; representation; lifetime; invariant; diagnostic; trade-off.

145.1 The problem and the mental model

Mutual exclusion protects a critical invariant by allowing one participant at a time. The protected region should be small, explicit, and paired with a clear ownership rule.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

145.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch145.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 145: Mutual Exclusion
#include <iostream>

int main() {
    std::cout << "Chapter 145: Mutual Exclusion" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

145.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

145.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses mutual exclusion and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

145.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

145.6 Summary and further study

Mutual Exclusion is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Locks

Learning objectives

- Explain the central problem addressed by Locks and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Locks; contract; representation; lifetime; invariant; diagnostic; trade-off.

146.1 The problem and the mental model

Locks provide synchronization protocols, not automatic correctness. Lock ordering, scope, contention, and exception-safe release determine whether a design remains live.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

146.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch146.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 146: Locks
#include <iostream>

int main() {
    std::cout << "Chapter 146: Locks" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

146.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

146.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses locks and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

146.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

146.6 Summary and further study

Locks is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Condition Variables

Learning objectives

- Explain the central problem addressed by Condition Variables and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Condition Variables; contract; representation; lifetime; invariant; diagnostic; trade-off.

147.1 The problem and the mental model

A condition variable lets a thread sleep until shared state may have changed. Waiting must always re-check a predicate because wakeups are not proofs that the condition holds.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

147.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch147.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 147: Condition Variables
#include <iostream>

int main() {
    std::cout << "Chapter 147: Condition Variables" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

147.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

147.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses condition variables and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

147.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

147.6 Summary and further study

Condition Variables is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Futures and Asynchronous Work

Learning objectives

- Explain the central problem addressed by Futures and Asynchronous Work and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Futures and Asynchronous Work; contract; representation; lifetime; invariant; diagnostic; trade-off.

148.1 The problem and the mental model

Futures represent a result that may become available later. Launch policy, exception propagation, cancellation, and destruction semantics must be documented.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

148.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch148.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 148: Futures and Asynchronous Work
#include <iostream>

int main() {
    std::cout << "Chapter 148: Futures and Asynchronous Work" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

148.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

148.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses futures and asynchronous work and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

148.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

148.6 Summary and further study

Futures and Asynchronous Work is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Atomic Operations

Learning objectives

- Explain the central problem addressed by Atomic Operations and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Atomic Operations; contract; representation; lifetime; invariant; diagnostic; trade-off.

149.1 The problem and the mental model

Atomics provide indivisible operations and synchronization orders for selected objects. They do not make a multi-object invariant automatically safe.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

149.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch149.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 149: Atomic Operations
#include <atomic>
#include <iostream>
#include <thread>

int main() {
    std::atomic<int> count{0};
    std::jthread first([&] { for (int i = 0; i < 1000; ++i) ++count; });
    std::jthread second([&] { for (int i = 0; i < 1000; ++i) ++count; });
    first.join(); second.join();
    std::cout << count.load() << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

149.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

149.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses atomic operations and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

149.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

149.6 Summary and further study

Atomic Operations is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

The C++ Memory Model

Learning objectives

- Explain the central problem addressed by The C++ Memory Model and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: The C++ Memory Model; contract; representation; lifetime; invariant; diagnostic; trade-off.

150.1 The problem and the mental model

The memory model describes visibility, ordering, happens-before, and data races. It explains why compiler and processor reordering can matter even when source statements look sequential.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

150.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch150.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 150: The C++ Memory Model
#include <iostream>

int main() {
    std::cout << "Chapter 150: The C++ Memory Model" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

150.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

150.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses the c++ memory model and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

150.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

150.6 Summary and further study

The C++ Memory Model is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Deadlocks and Livelocks

Learning objectives

- Explain the central problem addressed by Deadlocks and Livelocks and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Deadlocks and Livelocks; contract; representation; lifetime; invariant; diagnostic; trade-off.

151.1 The problem and the mental model

Deadlock stops progress through circular waiting; livelock continues activity without useful progress. Avoidance requires resource-ordering and progress reasoning, not just more threads.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

151.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch151.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 151: Deadlocks and Livelocks
#include <iostream>

int main() {
    std::cout << "Chapter 151: Deadlocks and Livelocks" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

151.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

151.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses deadlocks and livelocks and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

151.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

151.6 Summary and further study

Deadlocks and Livelocks is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Thread-Safe Design

Learning objectives

- Explain the central problem addressed by Thread-Safe Design and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Thread-Safe Design; contract; representation; lifetime; invariant; diagnostic; trade-off.

152.1 The problem and the mental model

Thread safety is a property of an interface under concurrent use. Immutable values, ownership transfer, message passing, and narrow synchronized boundaries often outperform shared mutable state as a design strategy.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

152.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch152.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 152: Thread-Safe Design
#include <iostream>

int main() {
    std::cout << "Chapter 152: Thread-Safe Design" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

152.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

152.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses thread-safe design and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

152.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

152.6 Summary and further study

Thread-Safe Design is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Parallel Algorithms

Learning objectives

- Explain the central problem addressed by Parallel Algorithms and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Parallel Algorithms; contract; representation; lifetime; invariant; diagnostic; trade-off.

153.1 The problem and the mental model

Parallel algorithms divide work under a policy, but speedup depends on independence, granularity, scheduling, memory bandwidth, and overhead.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

153.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch153.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 153: Parallel Algorithms
#include <iostream>

int main() {
    std::cout << "Chapter 153: Parallel Algorithms" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

153.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

153.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses parallel algorithms and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

153.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

153.6 Summary and further study

Parallel Algorithms is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Task-Based Concurrency

Learning objectives

- Explain the central problem addressed by Task-Based Concurrency and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 10.

Key terminology: Task-Based Concurrency; contract; representation; lifetime; invariant; diagnostic; trade-off.

154.1 The problem and the mental model

Task-based designs express units of work and dependencies rather than manually managing every thread. They improve composition when the scheduler and cancellation model are understood.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

154.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch154.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 154: Task-Based Concurrency
#include <iostream>

int main() {
    std::cout << "Chapter 154: Task-Based Concurrency" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

154.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

154.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses task-based concurrency and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

154.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

154.6 Summary and further study

Task-Based Concurrency is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART XI

Performance and Systems

Part 11: Performance and Systems

Measurement, hardware-aware reasoning, optimization, and choosing the right tool.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

What Performance Means

Learning objectives

- Explain the central problem addressed by What Performance Means and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: What Performance Means; contract; representation; lifetime; invariant; diagnostic; trade-off.

155.1 The problem and the mental model

Performance can mean latency, throughput, memory use, startup time, tail behavior, energy, or cost. A useful optimization starts by naming which one matters.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

155.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch155.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 155: What Performance Means
#include <iostream>

int main() {
    std::cout << "Chapter 155: What Performance Means" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

155.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

155.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses what performance means and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

155.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

155.6 Summary and further study

What Performance Means is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Profiling Before Optimization

Learning objectives

- Explain the central problem addressed by Profiling Before Optimization and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Profiling Before Optimization; contract; representation; lifetime; invariant; diagnostic; trade-off.

156.1 The problem and the mental model

Profiling locates observed cost in a workload. It prevents optimizing code that is elegant or suspicious but irrelevant to the actual bottleneck.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

156.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch156.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 156: Profiling Before Optimization
#include <iostream>

int main() {
    std::cout << "Chapter 156: Profiling Before Optimization" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

156.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

156.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses profiling before optimization and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

156.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

156.6 Summary and further study

Profiling Before Optimization is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

CPU, Cache, and Memory Locality

Learning objectives

- Explain the central problem addressed by CPU, Cache, and Memory Locality and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: CPU, Cache, and Memory Locality; contract; representation; lifetime; invariant; diagnostic; trade-off.

157.1 The problem and the mental model

Modern processors move data through a hierarchy. Locality is a useful model for why access patterns matter, but exact timings are architecture and workload dependent.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

157.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch157.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 157: CPU, Cache, and Memory Locality
#include <iostream>

int main() {
    std::cout << "Chapter 157: CPU, Cache, and Memory Locality" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

157.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

157.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses cpu, cache, and memory locality and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

157.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

157.6 Summary and further study

CPU, Cache, and Memory Locality is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Allocation Costs

Learning objectives

- Explain the central problem addressed by Allocation Costs and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Allocation Costs; contract; representation; lifetime; invariant; diagnostic; trade-off.

158.1 The problem and the mental model

Allocation has time, fragmentation, synchronization, and lifetime consequences. Fewer allocations can help, but pooling or custom allocators also add complexity and should be measured.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

158.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch158.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 158: Allocation Costs
#include <iostream>

int main() {
    std::cout << "Chapter 158: Allocation Costs" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

158.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

158.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses allocation costs and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

158.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

158.6 Summary and further study

Allocation Costs is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Data-Oriented Design

Learning objectives

- Explain the central problem addressed by Data-Oriented Design and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Data-Oriented Design; contract; representation; lifetime; invariant; diagnostic; trade-off.

159.1 The problem and the mental model

Data-oriented design chooses representation and iteration around access patterns. It complements object-oriented boundaries when data movement dominates runtime cost.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

159.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch159.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 159: Data-Oriented Design
#include <iostream>

int main() {
    std::cout << "Chapter 159: Data-Oriented Design" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

159.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

159.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses data-oriented design and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

159.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

159.6 Summary and further study

Data-Oriented Design is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Inlining and Optimization

Learning objectives

- Explain the central problem addressed by Inlining and Optimization and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Inlining and Optimization; contract; representation; lifetime; invariant; diagnostic; trade-off.

160.1 The problem and the mental model

Inlining is a compiler transformation, not a source-level command to make code fast. It can remove call overhead or expose optimization, but can also increase code size.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

160.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch160.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 160: Inlining and Optimization
#include <iostream>

int main() {
    std::cout << "Chapter 160: Inlining and Optimization" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

160.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

160.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses inlining and optimization and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

160.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

160.6 Summary and further study

Inlining and Optimization is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Compiler Optimization

Learning objectives

- Explain the central problem addressed by Compiler Optimization and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Compiler Optimization; contract; representation; lifetime; invariant; diagnostic; trade-off.

161.1 The problem and the mental model

Optimization flags change generated code under the language rules. They do not repair undefined behavior or make an invalid program valid.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

161.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch161.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 161: Compiler Optimization
#include <iostream>

int main() {
    std::cout << "Chapter 161: Compiler Optimization" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

161.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

161.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses compiler optimization and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

161.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

161.6 Summary and further study

Compiler Optimization is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Benchmark Design

Learning objectives

- Explain the central problem addressed by Benchmark Design and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Benchmark Design; contract; representation; lifetime; invariant; diagnostic; trade-off.

162.1 The problem and the mental model

A benchmark needs representative inputs, warm-up policy, measurement boundaries, repetition, and a way to avoid measuring dead-code elimination or setup accidentally.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

162.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch162.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 162: Benchmark Design
#include <iostream>

int main() {
    std::cout << "Chapter 162: Benchmark Design" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

162.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

162.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses benchmark design and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

162.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

162.6 Summary and further study

Benchmark Design is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Latency and Throughput

Learning objectives

- Explain the central problem addressed by Latency and Throughput and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Latency and Throughput; contract; representation; lifetime; invariant; diagnostic; trade-off.

163.1 The problem and the mental model

Latency measures the time for one operation; throughput measures completed work per unit time. A system can improve one while worsening the other.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

163.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch163.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 163: Latency and Throughput
#include <iostream>

int main() {
    std::cout << "Chapter 163: Latency and Throughput" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

163.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

163.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses latency and throughput and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

163.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

163.6 Summary and further study

Latency and Throughput is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Performance Portability

Learning objectives

- Explain the central problem addressed by Performance Portability and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Performance Portability; contract; representation; lifetime; invariant; diagnostic; trade-off.

164.1 The problem and the mental model

An optimization that helps one compiler or processor can harm another. Portable performance comes from a clear model, measurement on target environments, and modest assumptions.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

164.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch164.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 164: Performance Portability
#include <iostream>

int main() {
    std::cout << "Chapter 164: Performance Portability" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

164.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

164.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses performance portability and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

164.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

164.6 Summary and further study

Performance Portability is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Safe Optimization

Learning objectives

- Explain the central problem addressed by Safe Optimization and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: Safe Optimization; contract; representation; lifetime; invariant; diagnostic; trade-off.

165.1 The problem and the mental model

Safe optimization preserves contracts and makes the reason for a change reviewable. Correctness tests and before/after measurements are part of the optimization itself.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

165.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch165.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 165: Safe Optimization
#include <iostream>

int main() {
    std::cout << "Chapter 165: Safe Optimization" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

165.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

165.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses safe optimization and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

165.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

165.6 Summary and further study

Safe Optimization is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

When C++ Is an Appropriate Tool

Learning objectives

- Explain the central problem addressed by When C++ Is an Appropriate Tool and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: When C++ Is an Appropriate Tool; contract; representation; lifetime; invariant; diagnostic; trade-off.

166.1 The problem and the mental model

C++ is a strong choice when control over resource use, predictable performance, existing ecosystems, or native integration matters. Its costs include language complexity and a larger safety burden.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

166.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch166.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 166: When C++ Is an Appropriate Tool
#include <iostream>

int main() {
    std::cout << "Chapter 166: When C++ Is an Appropriate Tool" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

166.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

166.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses when c++ is an appropriate tool and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

166.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

166.6 Summary and further study

When C++ Is an Appropriate Tool is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

When Another Language May Be Better

Learning objectives

- Explain the central problem addressed by When Another Language May Be Better and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 11.

Key terminology: When Another Language May Be Better; contract; representation; lifetime; invariant; diagnostic; trade-off.

167.1 The problem and the mental model

A different language may reduce development time, memory-safety risk, or operational complexity. Choosing C++ should be a comparative engineering decision rather than an identity claim.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

167.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch167.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 167: When Another Language May Be Better
#include <iostream>

int main() {
    std::cout << "Chapter 167: When Another Language May Be Better" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

167.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

167.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses when another language may be better and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

167.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

167.6 Summary and further study

When Another Language May Be Better is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

PART XII

Advanced Language and Design

Part 12: Advanced Language and Design

Advanced mechanisms introduced through the problems they solve and the complexity they incur.

Design lens. Read the chapters in this part as answers to one question: what decision does this topic make visible, and what assumption must remain true for that decision to be safe? The code examples are intentionally small; the projects later combine the same ideas under realistic constraints.

Advanced Type-System Concepts

Learning objectives

- Explain the central problem addressed by Advanced Type-System Concepts and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Advanced Type-System Concepts; contract; representation; lifetime; invariant; diagnostic; trade-off.

168.1 The problem and the mental model

Types can encode ownership, alternatives, constraints, units, and state transitions. Strong types are valuable when their invariants justify the additional vocabulary.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

168.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch168.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 168: Advanced Type-System Concepts
#include <iostream>

int main() {
    std::cout << "Chapter 168: Advanced Type-System Concepts" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

168.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

168.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses advanced type-system concepts and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

168.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

168.6 Summary and further study

Advanced Type-System Concepts is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Perfect Forwarding

Learning objectives

- Explain the central problem addressed by Perfect Forwarding and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Perfect Forwarding; contract; representation; lifetime; invariant; diagnostic; trade-off.

169.1 The problem and the mental model

Perfect forwarding preserves value category and cv-qualification across a generic boundary. It is powerful for wrappers and factories but can make overload behavior harder to read.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

169.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch169.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 169: Perfect Forwarding
#include <iostream>

int main() {
    std::cout << "Chapter 169: Perfect Forwarding" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

169.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

169.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses perfect forwarding and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

169.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

169.6 Summary and further study

Perfect Forwarding is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Customization and Generic Interfaces

Learning objectives

- Explain the central problem addressed by Customization and Generic Interfaces and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Customization and Generic Interfaces; contract; representation; lifetime; invariant; diagnostic; trade-off.

170.1 The problem and the mental model

Customization points let generic code adapt to user types without hard-coding every type. A good customization protocol documents discovery, precedence, and semantic requirements.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

170.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch170.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 170: Customization and Generic Interfaces
#include <iostream>

int main() {
    std::cout << "Chapter 170: Customization and Generic Interfaces" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

170.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

170.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses customization and generic interfaces and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

170.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

170.6 Summary and further study

Customization and Generic Interfaces is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Allocators and Memory Resources

Learning objectives

- Explain the central problem addressed by Allocators and Memory Resources and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Allocators and Memory Resources; contract; representation; lifetime; invariant; diagnostic; trade-off.

171.1 The problem and the mental model

Allocators separate allocation policy from container logic; memory resources provide runtime allocation strategies through an interface. They are useful for arenas and locality, not as default decoration.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

171.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch171.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 171: Allocators and Memory Resources
#include <iostream>

int main() {
    std::cout << "Chapter 171: Allocators and Memory Resources" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

171.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

171.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses allocators and memory resources and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

171.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

171.6 Summary and further study

Allocators and Memory Resources is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Coroutines

Learning objectives

- Explain the central problem addressed by Coroutines and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Coroutines; contract; representation; lifetime; invariant; diagnostic; trade-off.

172.1 The problem and the mental model

Coroutines suspend and resume a computation while a library-defined promise type controls the surrounding protocol. The language supplies machinery; scheduling, lifetime, and cancellation remain design responsibilities.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

172.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch172.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 172: Coroutines
#include <iostream>

int main() {
    std::cout << "Chapter 172: Coroutines" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

172.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

172.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses coroutines and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

172.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

172.6 Summary and further study

Coroutines is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Advanced Ranges

Learning objectives

- Explain the central problem addressed by Advanced Ranges and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Advanced Ranges; contract; representation; lifetime; invariant; diagnostic; trade-off.

173.1 The problem and the mental model

Advanced ranges compose lazy transformations, filters, projections, and sentinels. The main risks are borrowed lifetimes, repeated work, and interfaces that hide expensive evaluation.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

173.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch173.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 173: Advanced Ranges
#include <iostream>

int main() {
    std::cout << "Chapter 173: Advanced Ranges" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

173.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

173.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses advanced ranges and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

173.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

173.6 Summary and further study

Advanced Ranges is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Metaprogramming

Learning objectives

- Explain the central problem addressed by Metaprogramming and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Metaprogramming; contract; representation; lifetime; invariant; diagnostic; trade-off.

174.1 The problem and the mental model

Metaprogramming generates or selects code at translation time. It can remove duplication and enforce invariants, but generated complexity must remain explainable to maintainers.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

174.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch174.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 174: Metaprogramming
#include <iostream>

int main() {
    std::cout << "Chapter 174: Metaprogramming" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

174.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

174.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses metaprogramming and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

174.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

174.6 Summary and further study

Metaprogramming is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Type Erasure

Learning objectives

- Explain the central problem addressed by Type Erasure and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Type Erasure; contract; representation; lifetime; invariant; diagnostic; trade-off.

175.1 The problem and the mental model

Type erasure stores values behind a uniform interface while hiding their concrete type. It trades some compile-time visibility for runtime flexibility and stable boundaries.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

175.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch175.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 175: Type Erasure
#include <functional>
#include <iostream>
#include <string>

int main() {
    std::function<std::string()> message = [] { return std::string{"erased"}; };
    std::cout << message() << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

175.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an ob-

served implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

175.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses type erasure and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

175.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

175.6 Summary and further study

Type Erasure is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Policy-Based Design

Learning objectives

- Explain the central problem addressed by Policy-Based Design and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Policy-Based Design; contract; representation; lifetime; invariant; diagnostic; trade-off.

176.1 The problem and the mental model

Policy-based design parameterizes a type with independent behaviors. It can create precise reusable components, but too many policies can make the resulting type difficult to understand.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

176.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch176.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 176: Policy-Based Design
#include <iostream>

int main() {
    std::cout << "Chapter 176: Policy-Based Design" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

176.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

176.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses policy-based design and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

176.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

176.6 Summary and further study

Policy-Based Design is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Architectural Patterns

Learning objectives

- Explain the central problem addressed by Architectural Patterns and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Architectural Patterns; contract; representation; lifetime; invariant; diagnostic; trade-off.

177.1 The problem and the mental model

Architecture allocates responsibilities and dependencies across a system. Patterns are vocabulary for recurring trade-offs, not templates that remove the need for domain reasoning.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

177.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch177.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 177: Architectural Patterns
#include <iostream>

int main() {
    std::cout << "Chapter 177: Architectural Patterns" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

177.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

177.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses architectural patterns and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

177.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

177.6 Summary and further study

Architectural Patterns is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Design Patterns in Modern C++

Learning objectives

- Explain the central problem addressed by Design Patterns in Modern C++ and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Design Patterns in Modern C++; contract; representation; lifetime; invariant; diagnostic; trade-off.

178.1 The problem and the mental model

Modern C++ changes the cost and shape of classic patterns through value semantics, lambdas, concepts, smart pointers, and standard components. Use the simplest mechanism that expresses the intent.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

178.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch178.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 178: Design Patterns in Modern C++
#include <iostream>

int main() {
    std::cout << "Chapter 178: Design Patterns in Modern C++" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

178.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

178.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses design patterns in modern c++ and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

178.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

178.6 Summary and further study

Design Patterns in Modern C++ is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Domain Modeling

Learning objectives

- Explain the central problem addressed by Domain Modeling and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Domain Modeling; contract; representation; lifetime; invariant; diagnostic; trade-off.

179.1 The problem and the mental model

Domain modeling gives important concepts explicit names, invariants, and operations. A useful model reflects decisions and constraints rather than mirroring every database column.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

179.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch179.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 179: Domain Modeling
#include <iostream>

int main() {
    std::cout << "Chapter 179: Domain Modeling" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

179.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

179.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses domain modeling and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

179.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

179.6 Summary and further study

Domain Modeling is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Stable Interfaces

Learning objectives

- Explain the central problem addressed by Stable Interfaces and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Stable Interfaces; contract; representation; lifetime; invariant; diagnostic; trade-off.

180.1 The problem and the mental model

A stable interface controls source, binary, behavioral, and operational compatibility. Compatibility is a budget with costs, not a property acquired by adding more abstraction.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

180.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch180.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 180: Stable Interfaces
#include <iostream>

int main() {
    std::cout << "Chapter 180: Stable Interfaces" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

180.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

180.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses stable interfaces and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

180.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

180.6 Summary and further study

Stable Interfaces is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Large-Codebase Architecture

Learning objectives

- Explain the central problem addressed by Large-Codebase Architecture and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Large-Codebase Architecture; contract; representation; lifetime; invariant; diagnostic; trade-off.

181.1 The problem and the mental model

Large systems need dependency direction, ownership of decisions, build boundaries, observability, and migration paths. Architecture is tested by how safely the system changes.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

181.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch181.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 181: Large-Codebase Architecture
#include <iostream>

int main() {
    std::cout << "Chapter 181: Large-Codebase Architecture" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

181.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence

of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

181.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses large-codebase architecture and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

181.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

181.6 Summary and further study

Large-Codebase Architecture is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Legacy-Code Modernization

Learning objectives

- Explain the central problem addressed by Legacy-Code Modernization and relate it to the C++ object, type, or program model.
- Use the idea in a small program, identify its main failure modes, and state one trade-off honestly.

Prerequisites: the preceding chapter, plus the vocabulary introduced in Part 12.

Key terminology: Legacy-Code Modernization; contract; representation; lifetime; invariant; diagnostic; trade-off.

182.1 The problem and the mental model

Modernization reduces risk while preserving necessary behavior. Characterization tests, seams, staged replacement, and explicit deprecation are more reliable than a wholesale rewrite by optimism.

The practical question is not merely “what syntax produces this result?” It is “what must be true before the operation, what changes during it, and what remains true afterward?” That question gives the topic a place in a larger program. A provisional beginner model is useful here, but it must be refined whenever the standard leaves a choice to the implementation or a library component adds a precondition.

182.2 A small working program

The complete standalone program for this chapter is stored at `examples/chapters/ch182.cpp` and is compiled by the verification script.

```
// Modern C++ - Chapter 182: Legacy-Code Modernization
#include <iostream>

int main() {
    std::cout << "Chapter 182: Legacy-Code Modernization" << "\n";
}
```

The program is intentionally complete: it includes its headers, defines ‘main’, and has no dependency on a hidden project file. The code is a teaching instrument, not a claim that the smallest program is the best production design. Larger interfaces should expose ownership, failure, and lifetime explicitly.

182.3 Rules, mistakes, and diagnostics

The language rule and the engineering guideline are related but different. The compiler can diagnose violations that are visible in the translation unit, while runtime errors can arise from invalid input, environmental failure, lifetime mistakes, data races, or an incorrect model. Begin debugging with the first reproducible symptom and reduce the case before changing several things at once.

Common mistakes include using a name before its declaration, assuming a representation is universal, ignoring a returned error, retaining a reference or view beyond its source lifetime, and relying on an observed implementation detail. When a rule is about undefined behavior, no particular output is evidence of correctness.

Undefined-behavior check. If an example in this chapter would be wrong because an object is out of lifetime, an access is out of bounds, a precondition is violated, or two unsynchronized operations race, the program is wrong even if it compiles. Run the relevant sanitizer and add a test that exercises the boundary.

182.4 Design, performance, and testing

Choose the simplest representation that makes the contract visible. Measure performance only after naming the workload and the metric. A local optimization that changes ownership, ordering, or error behavior is a design change and should be reviewed as such. A useful test checks an observable property, includes a boundary case, and fails with enough context to explain what was expected.

Worked micro-check. Suppose a program uses legacy-code modernization and a test fails only for an empty input or a repeated call. First write down the expected contract for that boundary, then make the smallest input that distinguishes valid absence from invalid use. The answer is not to add a guard blindly: decide whether the API should reject, represent absence, or preserve an invariant before the next call.

182.5 Exercises

1. Explain the topic in three sentences: one about purpose, one about a rule, and one about a limitation.
2. Modify the complete program so that it accepts one relevant input and reports one meaningful failure through the interface chosen in this chapter.
3. Write a test for the smallest boundary case and a second test for the case that would expose a lifetime, ownership, ordering, or complexity mistake.

Selected solution. A sound solution states the precondition before it changes the code, uses a named value or type for the result, checks the failure path, and tests both the ordinary case and the boundary. If the change cannot be explained without referring to an undocumented implementation detail, redesign the interface or document the assumption.

182.6 Summary and further study

Legacy-Code Modernization is best understood as a contract inside a larger system. Keep the distinction between standard requirement, implementation behavior, and engineering recommendation visible. Re-run the example, inspect its diagnostics, and compare the result with the corresponding project when the idea appears in a multi-file program. Further study should include the relevant standard-library reference, the C++ Core Guidelines, and the citations listed in the bibliography.

Project index

The codebase contains progressively larger examples rather than a disconnected collection of fragments. The main projects are:

- **Command-line calculator:** parsing, validation, and explicit error reporting.
- **Text analysis:** strings, character classification, maps, and deterministic output.
- **Task ledger:** a small persistent application with a library, command-line front end, tests, and CMake.
- **Graph and algorithm examples:** traversal, complexity, and testable data structures.
- **Concurrency example:** shared state, synchronization, and deterministic verification.
- **Filesystem search:** portable standard-library filesystem traversal with failure handling.

Glossary

ABI	Application binary interface: conventions that let separately compiled binary components interact.
Algorithm	A finite, precise procedure for transforming inputs into outputs or effects.
Borrow	A non-owning use of an object whose lifetime is managed elsewhere.
Compiler	A program that translates source code into another representation and diagnoses violations of language rules.
Dangling pointer	A pointer whose stored address no longer denotes a live object of the required kind.
Exception safety	Guarantees describing what remains true when an operation fails by throwing.
Invariant	A condition that a type or system promises to preserve at defined boundaries.
Lifetime	The interval during which an object exists and may be used as an object of its type.
Linker	A tool that combines object files and libraries into a program or library image.
Ownership	Responsibility for releasing a resource exactly once at the appropriate time.
Race condition	A behavior that depends on the relative timing of operations that are not correctly synchronized.
RAII	Resource acquisition is initialization: bind resource ownership to an object's lifetime.
Translation unit	A source file after preprocessing, treated as one unit by the compiler.
Undefined behavior	A case for which the C++ standard imposes no requirements on the program's behavior.

Bibliographic note

The bibliography combines standards, vendor documentation, recognized C++ references, and foundational computer-science texts. URLs were checked during production; the technical fact-check report records the access date and the claims for which each source was used.

Bibliography

- Hans-J. Boehm. Threads cannot be implemented as a library. *ACM SIGPLAN Notices*, 40(6):261–268, 2005. doi: 10.1145/1064978.1065042.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.
- Ulrich Drepper. What every programmer should know about memory. Technical report, Red Hat, Inc., 2007. URL <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>.
- GCC Project. C++ standards support in gcc, 2026. URL <https://gcc.gnu.org/projects/cxx-status.html>. Accessed 2026-07-17.
- Andrew Hunt and David Thomas. *The Pragmatic Programmer*. Addison-Wesley, 2 edition, 2019.
- International Organization for Standardization. Iso/iec 14882:2024: Programming languages – c++, 2024. URL <https://www.iso.org/standard/83626.html>. The published standard is commonly referred to as C++23.
- Nicolai M. Josuttis. *C++17: The Complete Guide*. Rheinwerk Computing, 2019.
- Kitware. Cmake documentation, 2026. URL <https://cmake.org/cmake/help/latest/>. Accessed 2026-07-17.
- Donald E. Knuth. *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley, 3 edition, 1997.
- John Lakos et al. *Large-Scale C++ Volume I: Process and Architecture*. Addison-Wesley, 2020.
- LLVM Project. C++ language status, 2026. URL https://clang.llvm.org/cxx_status.html. Accessed 2026-07-17.
- Scott Meyers. *Effective Modern C++*. O’Reilly Media, 2014.
- Microsoft. Microsoft c++ language conformance, 2026. URL <https://learn.microsoft.com/en-us/cpp/overview/visual-cpp-language-conformance>. Accessed 2026-07-17.
- John Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, 2018.
- Standard C++ Foundation. The standard, 2026a. URL <https://isocpp.org/std/the-standard>. Accessed 2026-07-17.
- Standard C++ Foundation. C++ standardization status, 2026b. URL <https://isocpp.org/std/status>. Accessed 2026-07-17.
- Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley, 4 edition, 2013.
- Bjarne Stroustrup. *A Tour of C++*. Addison-Wesley, 2 edition, 2018.
- Herb Sutter. *Exceptional C++*. Addison-Wesley, 2000.
- Herb Sutter and Bjarne Stroustrup. C++ core guidelines, 2026. URL <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>. Accessed 2026-07-17.

Index

- Abstract Interfaces, 125
- Access Control, 103
- Advanced Ranges, 369
- Advanced Type-System Concepts, 359
- Algorithm Selection, 257
- Algorithmic Thinking, 229
- Algorithms, 143
- Allocation Costs, 337
- Allocators and Memory Resources, 365
- Alternatives to Inheritance, 133
- API Design, 283
- Application Binary Interfaces, 297
- Architectural Patterns, 377
- Arithmetic and Operators, 31
- Arrays, 65
- Assertions, 197
- Associative Containers, 151
- Asymptotic Complexity, 233
- Atomic Operations, 317

- Benchmark Design, 345
- Benchmarking, 259
- Binary Representation, 59

- Characters, Text, Encoding, and Unicode, 63
- Class Templates, 175
- Classes and Objects, 101
- CMake, 273
- Code Review, 289
- Coding Conventions, 285
- Comparisons and Boolean Logic, 33
- Compile-Time Programming, 183
- Compiler Optimization, 343
- Compilers, Linkers, Libraries, and Build Processes, 9
- Composition, 119
- Concepts and Constraints, 179
- Condition Variables, 313
- const Correctness, 113
- Constants and constexpr, 51
- constexpr Evaluation, 185
- Constructors, 105
- Containers, 139
- Continuous Integration, 291
- Copy and Move Semantics, 91
- Coroutines, 367

- CPU, Cache, and Memory Locality, 335
- Cross-Platform Development, 295
- Customization and Generic Interfaces, 363

- Data-Oriented Design, 339
- Deadlocks and Livelocks, 321
- Debuggers, 215
- Defensive Programming, 225
- Dependency Management, 275
- Design Patterns in Modern C++, 379
- Destructors, 107
- Documentation, 281
- Domain Modeling, 381
- Dynamic Programming, 253
- Dynamic Storage, 79

- Enumerations, 53
- Error Codes, 201
- Exception Safety, 205
- Exceptions, 199
- Expected-Style Error Handling, 203

- File Input and Output, 165
- Filesystem Operations, 169
- Formatting, 167
- Function Objects, 147
- Function Parameters and Return Values, 45
- Function Templates, 173
- Functions, 43
- Fundamental Types, 27
- Futures and Asynchronous Work, 315

- Generic Algorithms, 189
- Graphs, 251
- Greedy Algorithms, 255

- Hardware, Software, and Operating Systems, 5
- Hash Tables, 249
- Heaps, 247

- Inheritance, 121
- Inlining and Optimization, 341
- Input and Output, 35
- Installing a C++ Development Environment, 11
- Integers, Floating-Point Values, and Precision, 61
- Integration Testing, 209

- Interoperability with C, 299
- Interoperability with Other Languages, 301
- Invariants, 109
- Iterators, 141
- Lambda Expressions, 149
- Large-Codebase Architecture, 385
- Latency and Throughput, 347
- Legacy-Code Modernization, 387
- Libraries, 269
- Linked Structures, 241
- Literals, 29
- Locks, 311
- Logging, 221
- Measuring Time and Space, 231
- Member Functions, 111
- Metaprogramming, 371
- Modules and Their Support Status, 271
- Multiple Inheritance, 131
- Mutual Exclusion, 309
- Namespaces, 55
- Null Pointers, 77
- Object Lifetime, 83
- Object Slicing, 129
- One Definition Rule, 265
- Operator Overloading, 117
- Optional Values and Variants, 157
- Organizing Source and Header Files, 263
- Overloading and Default Arguments, 47
- Package Managers, 277
- Parallel Algorithms, 325
- Perfect Forwarding, 361
- Performance Portability, 349
- Pointers, 75
- Policy-Based Design, 375
- Portability, 293
- Processes and Threads, 305
- Profiling Before Optimization, 333
- Program Structure, 21
- Property-Based Testing, 211
- Race Conditions, 307
- RAII, 89
- Random-Number Facilities, 163
- Ranges and Views, 145
- Reading Compiler Diagnostics, 17
- Recursion, 235
- Refactoring, 287
- References, 73
- Repetition with for, while, and do-while, 39
- Reproducing and Minimizing Bugs, 223
- Resource Ownership, 85
- Runtime Polymorphism, 123
- Safe Optimization, 351
- Sanitizers, 217
- Scope, Lifetime, and Name Visibility, 41
- Searching, 237
- Selection with if and switch, 37
- Separate Compilation, 267
- Smart Pointers, 87
- Sorting, 239
- Source Code, Machine Code, and Executable Files, 7
- Stable Interfaces, 383
- Stack and Free-Store Concepts, 81
- Stacks and Queues, 243
- Stacks, Queues, and Priority Queues, 155
- Statements, Expressions, and Blocks, 23
- Static Analysis, 219
- Static Members, 115
- std::array, 67
- std::string and std::string_view, 71
- std::vector, 69
- Structures, 99
- Syntax, Linker, Logic, and Runtime Errors, 195
- Task-Based Concurrency, 327
- Template Argument Deduction, 177
- Template Errors and Diagnostics, 191
- Terminals, Editors, IDEs, and File Systems, 13
- Test Doubles, 213
- The C++ Memory Model, 319
- The Purpose and Structure of the Standard Library, 137
- Thread-Safe Design, 323
- Time and Clocks, 161
- Trees, 245
- Tuples, 159
- Type Deduction with auto, 49
- Type Erasure, 373
- Type Traits, 187
- Undefined, Unspecified, and Implementation-Defined Behavior, 95
- Unit Testing, 207
- Unordered Containers, 153
- Value Categories, 93
- Variables and Initialization, 25
- Variadic Templates, 181
- Version Control with Git, 279

Virtual Functions, 127

What a Computer Program Is, 3

What Performance Means, 331

When Another Language May Be Better, 355

When C++ Is an Appropriate Tool, 353

Writing, Compiling, and Running the First
Program, 15